



ICRobotics 2025-26

Programming Guide

Introduction

This document goes through making a mock robot program to solve some common FRC problems. It aims to introduce concepts in the order that they can be best digested by a new programmer rather than in the order that you might implement them on a real robot.

The difficulty ramps up fairly quickly, but you are not expected to work through everything on your own. This guide is best used in conjunction with attending the weekly IC workshop nights where you'll be able to learn from mentors and other students.

The code you make using this guide will unfortunately not be run on a real robot, it is intended to be run in simulation only.

Some terms used throughout the guide have links to external sources, you don't have to follow and read most of these links, but it might help if you feel more explanation on a certain topic would be useful. These links are [highlighted blue with an underline](#), you can select them and then click the link in the box that appears.

Installing the software

Follow [these instructions](#) to install WPILib and its tools.

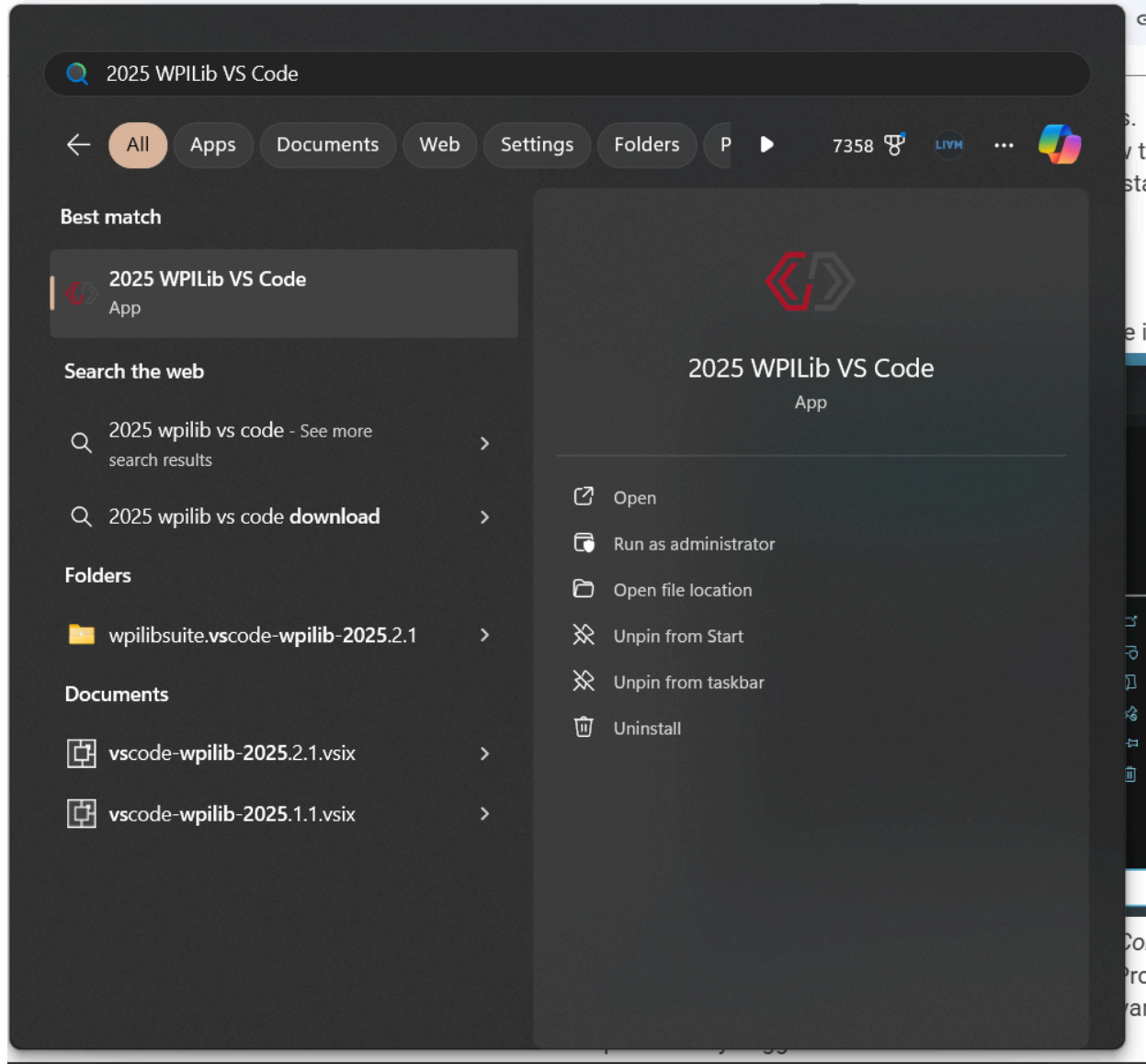
Make sure to scroll down a little further and also follow the **Additional C++ Installation for Simulation** step.

While running the installer, choose *Everything* as the install mode and click *Install for all Users*.

Starter Code

Making a new project

Open the newly installed WPILib VS Code (this example is from 2025):



Follow [these instructions](#) up to and including the *C++ Configurations* step. When making your project, use the following options (Replacing the Base Folder and Project Name with whatever you want):

Note: We are using the Command Robot Skeleton (Advanced) template, rather than the Getting Started Example like they suggest in the instructions.

Welcome to WPILib New Project Creator

template cpp Command Robot Skeleton (Advanced)

Base Folder. Select a base folder to place the new project into.

where/ever/you/want

Select a new project folder

Project Name

WhateverYouWant

Create a new folder? ☒

This creates a new folder at Base Folder\Project Name. Highly recommended to be checked. Otherwise the folder will be placed at Base Folder and not utilize the Project Name

Team Number

5584

Enable Desktop Support ☒

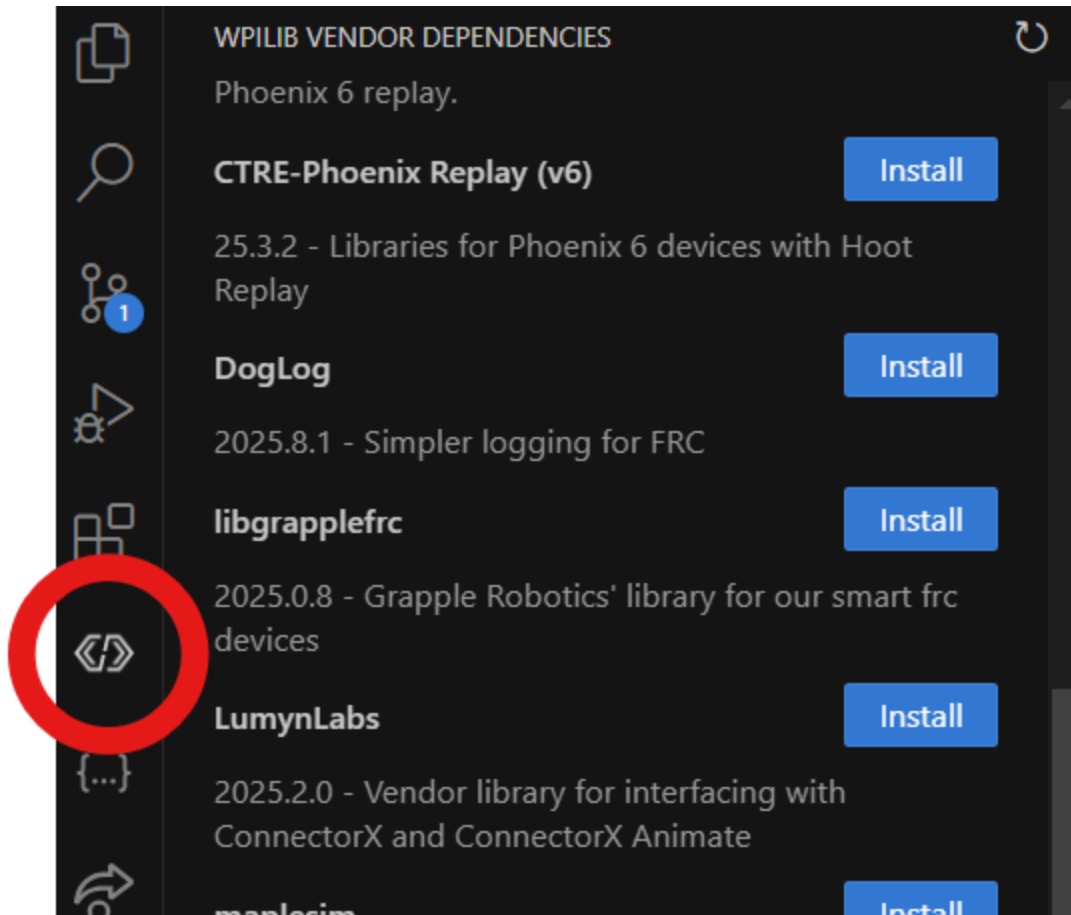
This is needed for simulation and unit testing support, however there are some cases where this will do some unexpected things during build. In addition, not all vendor libraries support desktop. This option can be set with the command "WPILib: Set Desktop Support" at any time.

Generate Project

Installing Vendor Libraries

When we want to control hardware like a motor, we often need to download code (sometimes called a library or dependency) that the manufacturer provides. In this Starter Code example, we will only install one library: RevLib. This is from [Rev Robotics](#) and allows us to run their [Spark](#) motor controllers, which we use a lot in IC.

1. In the sidebar in VSCode, open the WPILib Vendor Dependencies tab



2. Find REVLib and install using the blue button.

WPILIB VENDOR DEPENDENCIES

Update All

> INSTALLED DEPENDENCIES 6

< AVAILABLE DEPENDENCIES 14

AdvantageKit

4.1.2 - Simulation replay and logging framework

Install

ChoreoLib

2025.0.3 - Library for Choreo trajectory following

Install

CTRE-Phoenix (v5)

5.35.1 - Libraries for Phoenix 5 devices

Install

CTRE-Phoenix (v5) Replay Compatibility

5.35.1 - Compatibility library for Phoenix 5 when using Phoenix 6 replay.

Install

CTRE-Phoenix Replay (v6)

25.3.2 - Libraries for Phoenix 6 devices with Hoot Replay

Install

DogLog

2025.8.1 - Simpler logging for FRC

Install

libgrapplefrc

2025.0.8 - Grapple Robotics' library for our smart frc devices

Install

LumynLabs

2025.2.0 - Vendor library for interfacing with ConnectorX and ConnectorX Animate

Install

maplesim

0.3.8 - FRC Java Robot Simulation using a physics engine

Install

PlayingWithFusion

2025.01.23 - Library for all Playing with Fusion devices including Venom smart motors, time of flight sensors and more

Install

ReduxLib

Install

REVLlib

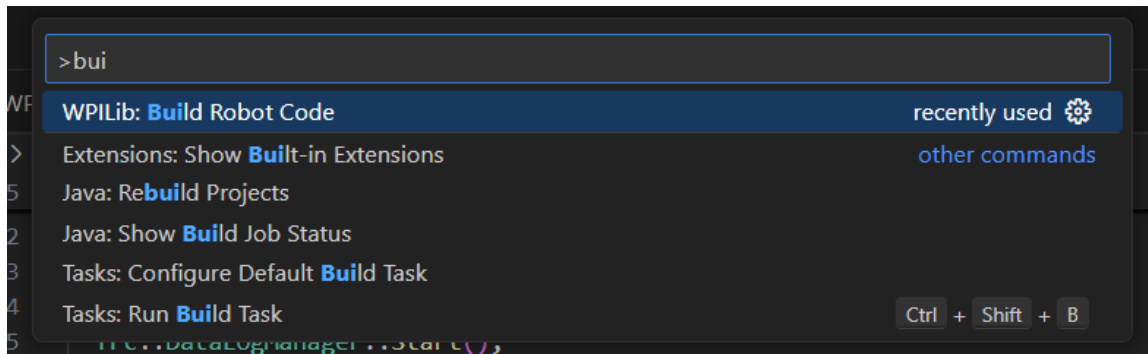
2025.0.3 - Library for all REV devices including SPARK

Install

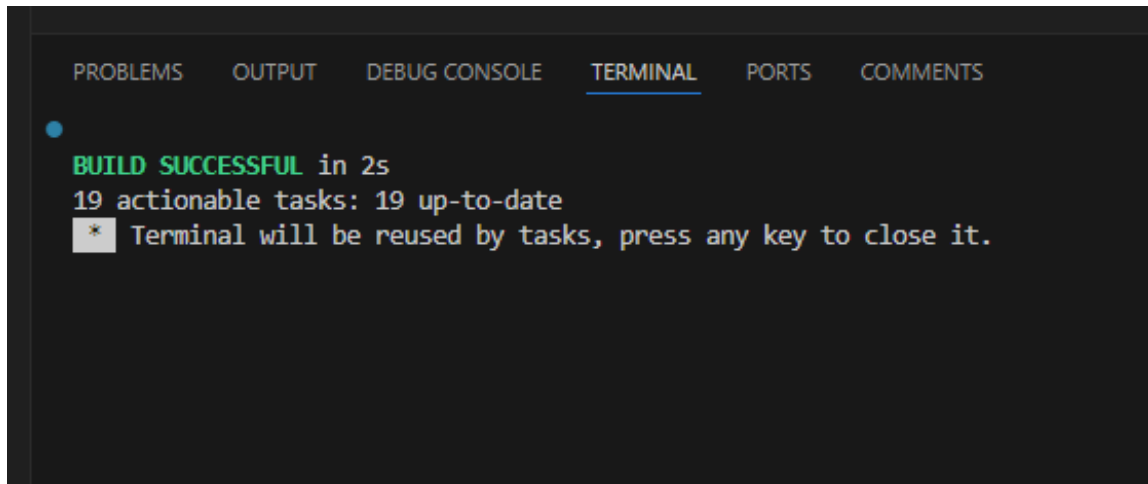
ThriftyLib

Install

3. To make sure the install has worked, we should “build” the code. This runs a process to convert your human readable code into machine readable instructions. To do this, open VSCode’s command palette with Ctrl+Shift+P and run the *WPILib: Build Robot Code* command.

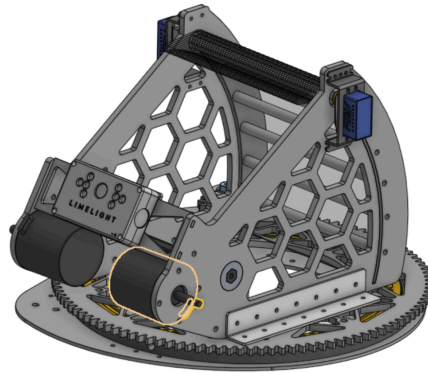


4. The terminal should open at the bottom of the screen, run a build process and eventually show “BUILD SUCCESSFUL” in green. This process can sometimes take several minutes.



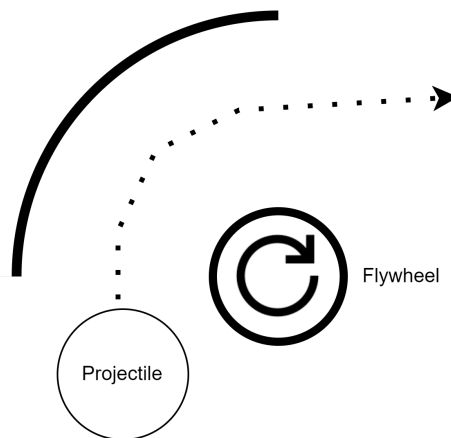
Making a shooter subsystem

Our goal now will be to write some code to run a ball shooter like one of these:



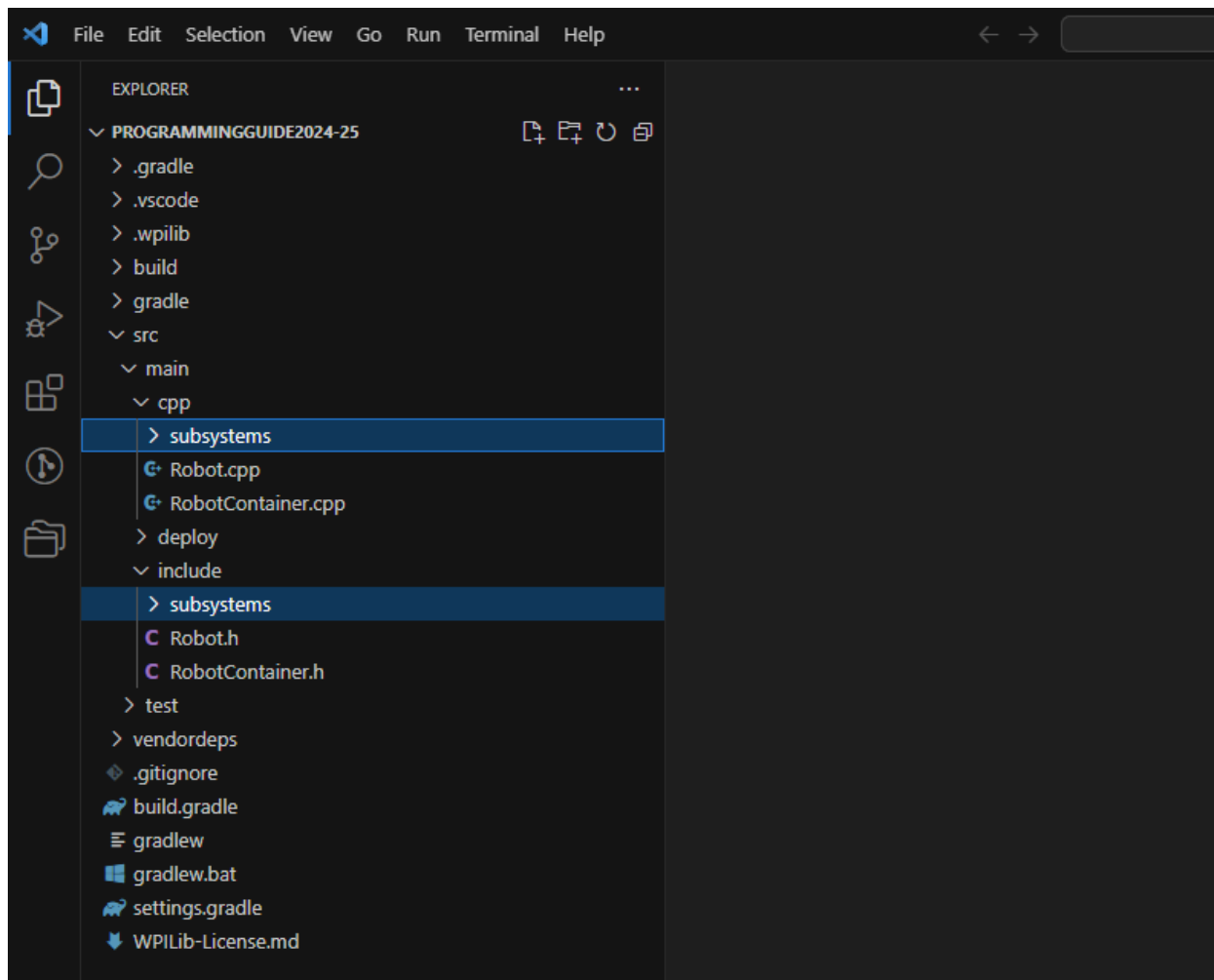
[Video of FRC 95's 2020 Shooter](#)

Shooting balls is a very common problem in FRC, it normally involves spinning one or more large wheels (aka flywheels) very fast, then moving a ball into contact with the wheels to fire it toward a goal. In the above pictures, balls are fed in from the bottom while the shooter wheels at the front spin to fire the balls forward and upward.

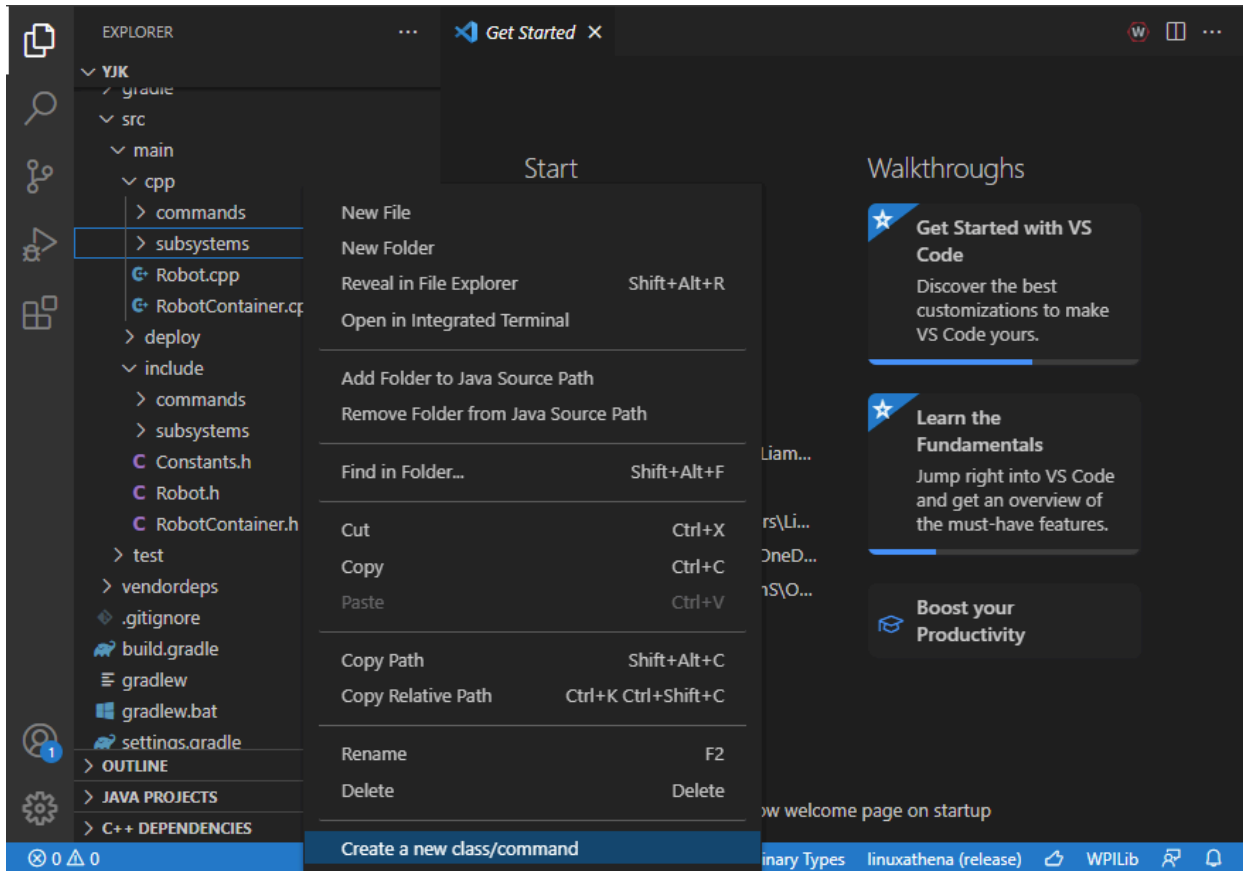


The shooter is a [subsystem](#), which is a part of the robot with its own actuators, sensors and structural components. We organise our code into subsystems, so let's make one for the shooter.

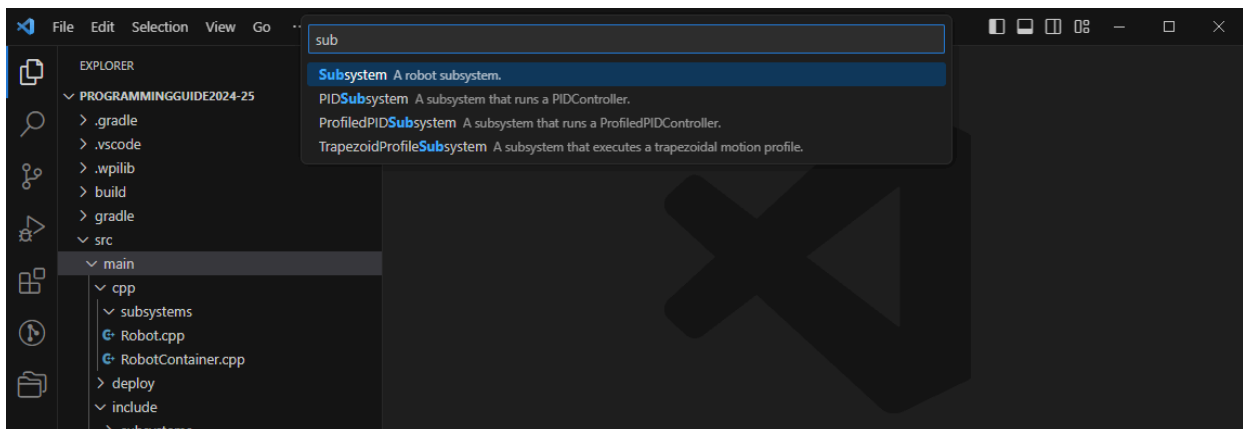
On the left of VSCode, right click the cpp folder under src/main and select "New Folder...", call it "subsystems". Create another subsystems folder under the src/main/include folder:



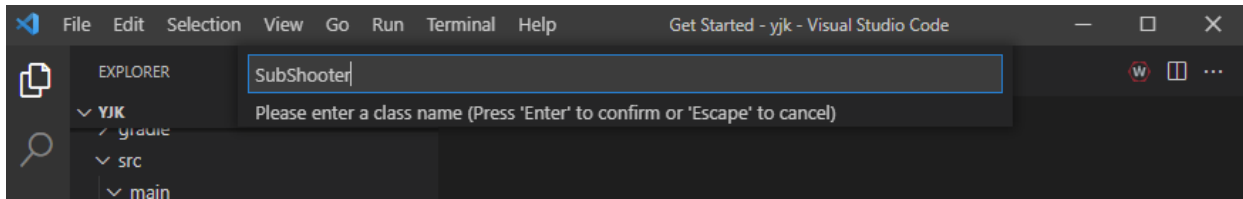
We have a folder to store our subsystems, now let's make a subsystem for the shooter. Right click the src/main/cpp/subsystems folder and select Create new class/command.



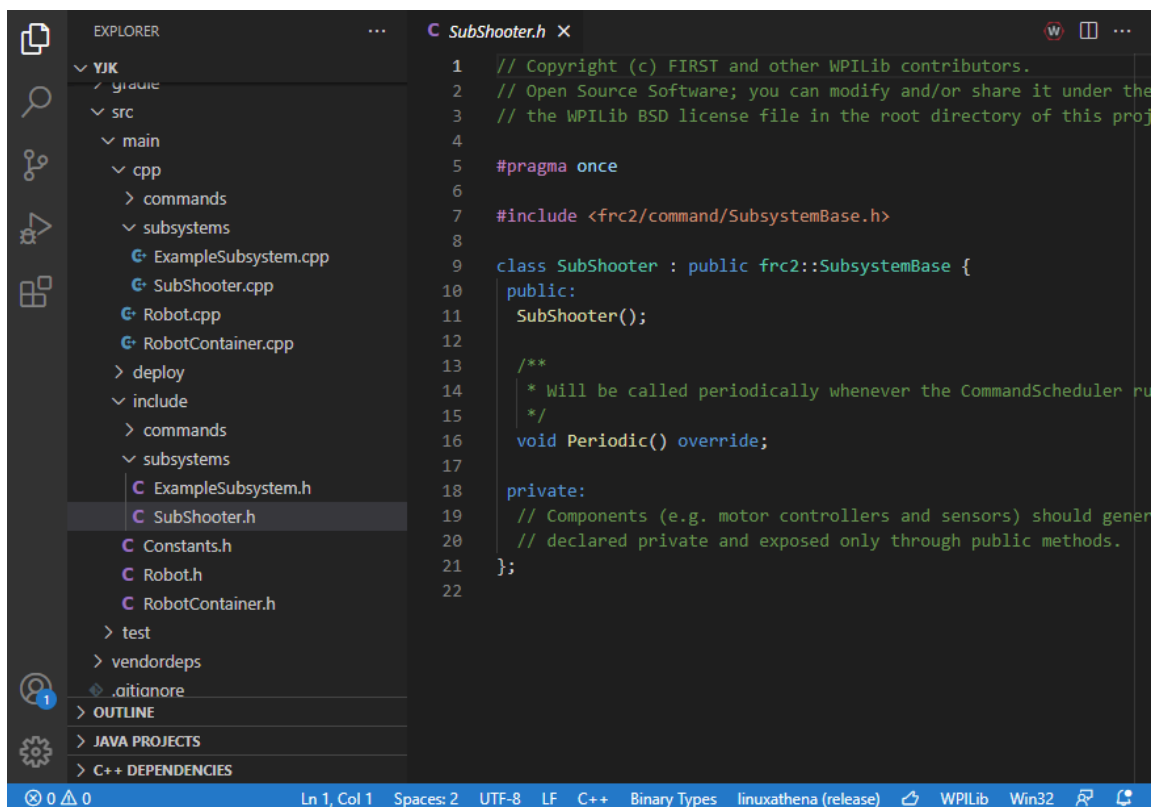
Select Subsystem



Enter SubShooter as the name. As a convention, we always start our subsystem names with “Sub”. This helps differentiate them from Commands, which we will be making later.



Open the new SubShooter.h file that has been created in src/main/include/subsystems. This is a header file (ends with .h), which is one half of the subsystem we just made. The other half is the body file (ends in .cpp). While clicked into SubShooter.h's contents on the right side of VS Code, press Alt+O to switch to SubShooter's body file. Press Alt+O again to swap back.



Header files are where we declare **what** a subsystem has and can do. Body files are where we define **how** the subsystem does those things.

In our case, the SubShooter:

- Has one motor (controlled by a Spark Flex motor controller), and
- Can Start spinning, and
- Can Stop spinning.

Our next goal is to make an object in our code that can control a Spark Flex motor controller. Making objects in C++ code is done by writing the **Type** of the object you want to create, then the **Name**, and finally any **inputs** that the object needs. For example, making an object to represent a pen might look like this:

```
Pen myPen{"blue"};
```

This creates an object of the `Pen` type, called `myPen` and the word `"blue"` has been given to it as input. Sometimes we would also describe this line of code as “Constructing a Pen object”, or even “Making a Pen”.

Types must exist in code before they can be used, a few types are available out of the box (like [integers](#), [booleans](#) and [characters](#)), but many of the more complex types need to be included from other sources first.

Let’s include the SparkFlex type that Rev Robotics provides as part of their Vendor Library. The highlighted line of code in the header file below shows how to do that. Follow along by typing out these highlighted sections in your own code.

```
5  #pragma once
6
7  #include <frc2/command/SubsystemBase.h>
8  #include <rev/SparkFlex.h>
9
10 class SubShooter : public frc2::SubsystemBase {
```

And then constructing a `rev::CANSparkMax` object down lower in the header file like so:

```
19 private:
20     // Components (e.g. motor controllers and sensors) should generally be
21     // declared private and exposed only through public methods.
22     rev::spark::SparkFlex _shooterMotor{1, rev::spark::SparkLowLevel::MotorType::kBrushless};
23 };
24
```

Here, we are naming our new Spark Flex “_shooterMotor”. It can be named whatever you like, but try to follow the conventions for variable naming in our [Software Standards](#).

In our inputs to the Spark Flex, the number 1 denotes that this Spark Flex has a CAN ID of 1. We use a [CAN Bus](#) to send signals to many of the devices on our robots, and every one needs a unique CAN ID.

The `rev::spark::SparkLowLevel::MotorType::kBrushless` input tells the Spark Flex that the motor attached to it is a brushless motor ([rather than a brushed motor](#)). In IC, we almost always use brushless motors.

We have now added all the required electronics to our subsystem. Next we need to tell our SubShooter what it can do by declaring a [function](#).

Functions are blocks of code that can be called upon to run later. You can identify functions from the () parentheses that must always appear at the end of their names.

```
8  #include <rev/SparkFlex.h>
9  #include <frc2/command/Commands.h>
10
11  class SubShooter : public frc2::SubsystemBase {
12  public:
13      SubShooter();
14
15      /**
16       * Will be called periodically whenever the CommandScheduler runs.
17       */
18      void Periodic() override;
19      frc2::CommandPtr SpinFlywheel();
20  }
```

Here, we make a function called `SpinFlywheel()` that creates an `frc2::CommandPtr` (pronounced “Command Pointer”), which is an action that our robot can perform. While in reality `SpinFlywheel` is a function that creates a command, when talking about these kinds of functions we often simplify things and just call them commands.

We made the Spark Flex object under the “private:” heading and the function under the “public:” heading. This is because the Spark Flex object does not need to be directly used by any code outside our shooter subsystem, whereas functions will need to be accessed from elsewhere, so they need to be public.

Next we need to tell the shooter how to spin its flywheel by giving our function a *definition* in the body file (ending in `.cpp`). Remember `Alt+O` can swap to the body file quickly. Swap over to the body file and add the highlighted section:

```

src > main > cpp > subsystems > SubShooter.cpp > SpinFlywheel()
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "subsystems/SubShooter.h"
6
7 SubShooter::SubShooter() = default;
8
9 // This method will be called once per scheduler run
10 void SubShooter::Periodic() {}
11
12 frc2::CommandPtr SubShooter::SpinFlywheel(){
13
14 }
15
16
17

```

Every line of code in between the { } braces of a function will run when we run (aka "call") that function. Later on we'll call this function when a human driver presses a button on their controller.

We still need to fill our functions with some code to control our Spark Flex:

```

11
12 frc2::CommandPtr SubShooter::SpinFlywheel() {
13     return StartEnd(
14         [this]{
15             _shooterMotor.Set(0.8); // On Start
16         },
17         [this]{
18             _shooterMotor.Set(0); // On End
19         }
20     );
21 }
22

```

These lines make a StartEnd command that will run one thing when it starts and another thing when it ends. On start we are telling the Spark Flex to spin the motor attached to it at 80% power, and on end we are telling it to spin at 0% power. Setting the motor back to 0% power is important as otherwise it would stay spinning even when the command stops. A command should almost always "clean itself up" in that way.

`_shooterMotor.set(0)` makes the motor stay still, `_shooterMotor.set(1)` spins the motor at full power. In general, the more power you put to a motor,

the faster it will spin. It's worth questioning yourself at this point, how do you think we could spin the motor backwards?

There is one more bit of housekeeping we need to do in our Subsystem to make it easily accessible from other parts of the code. Jump back to the header file with Alt+O and add this GetInstance() function near the top.

```
8  #include <frc/SparkFlex.h>
9
10 class SubShooter : public frc2::SubsystemBase {
11 public:
12     static SubShooter& GetInstance() {static SubShooter inst; return inst;}
13     SubShooter();
14
15     /**
16      * Will be called periodically whenever the CommandScheduler runs
```

This effectively turns our Subsystem into a [Singleton](#). This is an in-depth topic, but for the purposes of our Starter Code example, it is simply helping us access our Subsystem from elsewhere in the program.

The Shooter Subsystem is now complete! It contains a Spark Flex motor controller and is capable of spinning and stopping. In the next section, we will look at how a human driver can operate this subsystem.

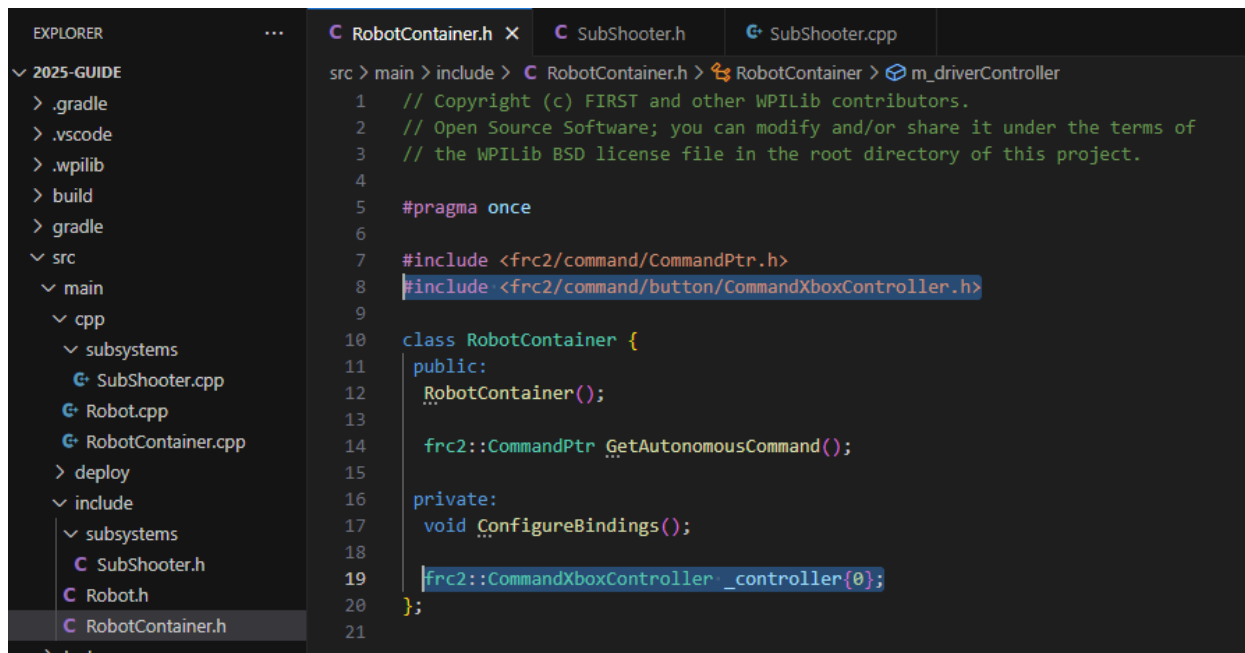
```
C SubShooter.h X SubShooter.cpp
src > main > include > subsystems > C SubShooter.h > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #pragma once
6
7 #include <frc2/command/SubsystemBase.h>
8 #include <rev/SparkFlex.h>
9 #include <frc2/command/Commands.h>
10
11 class SubShooter : public frc2::SubsystemBase {
12 public:
13     static SubShooter& GetInstance() {static SubShooter inst; return inst;};
14     SubShooter();
15
16     /**
17      * Will be called periodically whenever the CommandScheduler runs.
18      */
19     void Periodic() override;
20     frc2::CommandPtr SpinFlywheel();
21
22 private:
23     // Components (e.g. motor controllers and sensors) should generally be
24     // declared private and exposed only through public methods.
25     rev::SparkFlex _shooterMotor{1, rev::spark::SparkLowLevel::MotorType::kBrushless};
26 };
27
```

```
C SubShooter.h X SubShooter.cpp X
src > main > cpp > subsystems > C SubShooter.cpp > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "subsystems/SubShooter.h"
6
7 SubShooter::SubShooter() = default;
8
9 // This method will be called once per scheduler run
10 void SubShooter::Periodic() {}
11
12 frc2::CommandPtr SubShooter::SpinFlywheel() {
13     return StartEnd(
14         [this] {
15             _shooterMotor.Set(0.8); // On Start
16         },
17         [this] {
18             _shooterMotor.Set(0); // On End
19         }
20     );
21 }
22
```

Running commands on buttons

In order for a subsystem's commands to be accessible to the robot's driver, we need to hook the command up to the buttons of an Xbox controller. This is done inside the [RobotContainer](#). RobotContainer is a section of code that connects *Triggers* to *Commands*. For example, The A button on an Xbox controller can be a trigger, and our recently created SpinFlywheel() is a command.

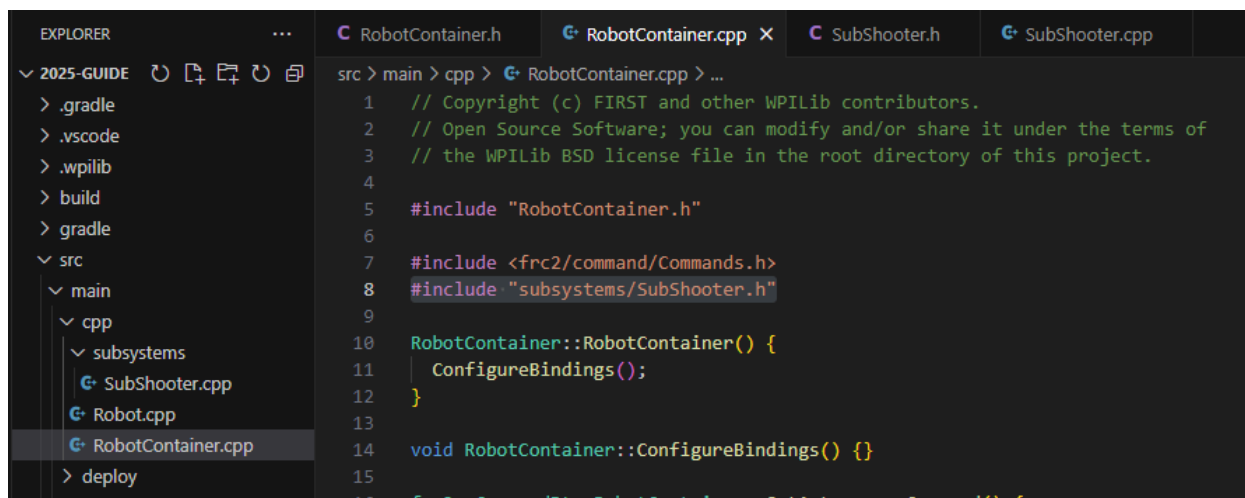
Open the RobotContainer.h file from the explorer pane on the left of VSCode and make a CommandXboxController by including <frc2/command/button/XboxController.h> up the top and making an frc2::CommandXboxController object called _controller down the bottom.

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project structure with folders like .gradle, .vscode, .wpilib, build, gradle, src, main, cpp, subsystems, and include. The file RobotContainer.h is selected. The main editor area shows the content of RobotContainer.h, which includes a copyright notice, a pragma once, and includes for frc2/command/CommandPtr.h and frc2/command/button/CommandXboxController.h. The class RobotContainer is defined with public methods RobotContainer() and GetAutonomousCommand(), and a private method ConfigureBindings(). The line frc2::CommandXboxController _controller{0}; is highlighted in blue.

```
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #pragma once
6
7 #include <frc2/command/CommandPtr.h>
8 #include <frc2/command/button/CommandXboxController.h>
9
10 class RobotContainer {
11 public:
12     RobotContainer();
13
14     frc2::CommandPtr GetAutonomousCommand();
15
16 private:
17     void ConfigureBindings();
18
19     frc2::CommandXboxController _controller{0};
20 };
21
```

As an input to the _controller object, we enter the number zero. This denotes that the controller we are interested in was the first one plugged into the computer. Importantly, the controller count starts at 0 not 1, which you will see [happen a lot in computer code](#). You can have up to six controllers plugged into the computer at a time, and pass in an ID from 0 to 5 to access any one of them.

Jump over to the RobotContainer body (.cpp) file and include your subsystem from subsystems/SubShooter.h.

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows the same project structure as the previous image. The file RobotContainer.cpp is selected. The main editor area shows the content of RobotContainer.cpp, which includes a copyright notice, a pragma once, and includes for RobotContainer.h, frc2/command/Commands.h, and subsystems/SubShooter.h. The class RobotContainer is defined with a constructor that calls ConfigureBindings(), and a method ConfigureBindings() that is currently empty. The line frc2::CommandPtr RobotContainer::GetAutonomousCommand() { is partially visible at the bottom.

```
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "RobotContainer.h"
6
7 #include <frc2/command/Commands.h>
8 #include "subsystems/SubShooter.h"
9
10 RobotContainer::RobotContainer() {
11     ConfigureBindings();
12 }
13
14 void RobotContainer::ConfigureBindings() {}
15
16 frc2::CommandPtr RobotContainer::GetAutonomousCommand() {

```

Notice we surround the file path of our subsystem's include using " " quotation marks rather than < > chevrons. This is because SubShooter.h is a file we created ourselves rather than one provided to us from outside the project. In reality both " " quotations and < > chevrons work for any includes, but they start their search for the file to include in different locations, so using " " quotations to tell the computer to start its search with our project's local files makes it minutely more efficient. More importantly, this makes it clear to anyone reading our code which files are our own and which are provided from 3rd parties.

With these included, we can go ahead and link our SpinFlywheel command to the Xbox controller's A button. Write the below code inside the existing ConfigureButtonBindings() function.

```
11 |   ConfigureBindings();  
12 | }  
13 |  
14 | void RobotContainer::ConfigureBindings() {  
15 |     _controller.A().WhileTrue(SubShooter::GetInstance().SpinFlywheel());  
16 | }  
17 |  
18 | frc2::CommandPtr RobotContainer::GetAutonomousCommand() {
```

We have now successfully linked our SpinFlywheel command to the A button! When A is first pressed, SpinFlywheel will start (setting the motor power to 0.8) and when A is released, SpinFlywheel will be canceled (setting the motor power to 0).

Testing in simulation

The code we have written so far is enough to work on a real robot. There is however one more thing we need to do in order to see our shooter motor's output in simulation. Head back to SubShooter.cpp, include SmartDashboard.h and add this to the periodic function:

```
SubShooter.h SubShooter.cpp X RobotContainer.cpp RobotContainer.h
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "subsystems/SubShooter.h"
6 #include <frc/smartdashboard/SmartDashboard.h>
7
8 SubShooter::SubShooter() = default;
9
10 // This method will be called once per scheduler run
11 void SubShooter::Periodic() {
12     frc::SmartDashboard::PutNumber("shooter speed", _shooterMotor.Get());
13 }
14
```

SmartDashboard is a file of code that lets us send information from our robot back to a dashboard that the human driving the robot can see. In this case, we are putting a number to the dashboard with the label “shooter speed” and a value equal to `_shooterMotor.Get()`.

`.Get()` is a function of the Spark motor controller that retrieves (aka “returns”) the amount of power being sent to the motor, measured as 0.0 being no power and 1.0 being full power forward. Thus, if the shooter motor was moving at 100% power, you could think of the `_shooterMotor.Get()` part effectively being replaced with the number 1, which would look like:

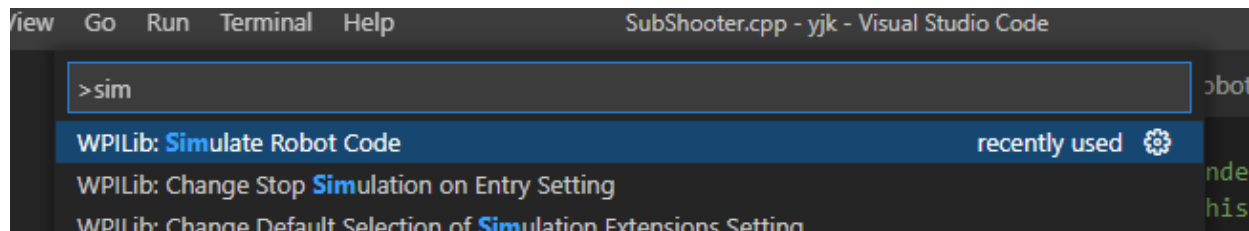
```
(Don't actually write this, it's just a
way to think about what's happening)
frc::SmartDashboard::PutNumber
r("shooter speed", 1);
```

Likewise, if our shooter motor was moving at 50% power, you could think of it as looking like:

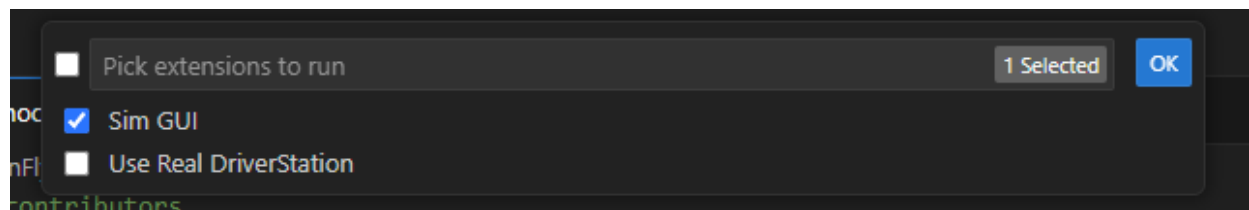
```
frc::SmartDashboard::PutNumber
r("shooter speed", 0.5);
```

With this line of code, the shooter power is now visible to the operator on the dashboard. We can also view all the dashboard numbers in simulation.

We now have all the code we need to run a test. Open the command pallet with `Ctrl+Shift+P` and run the *WPILib: Simulate Robot Code* command.

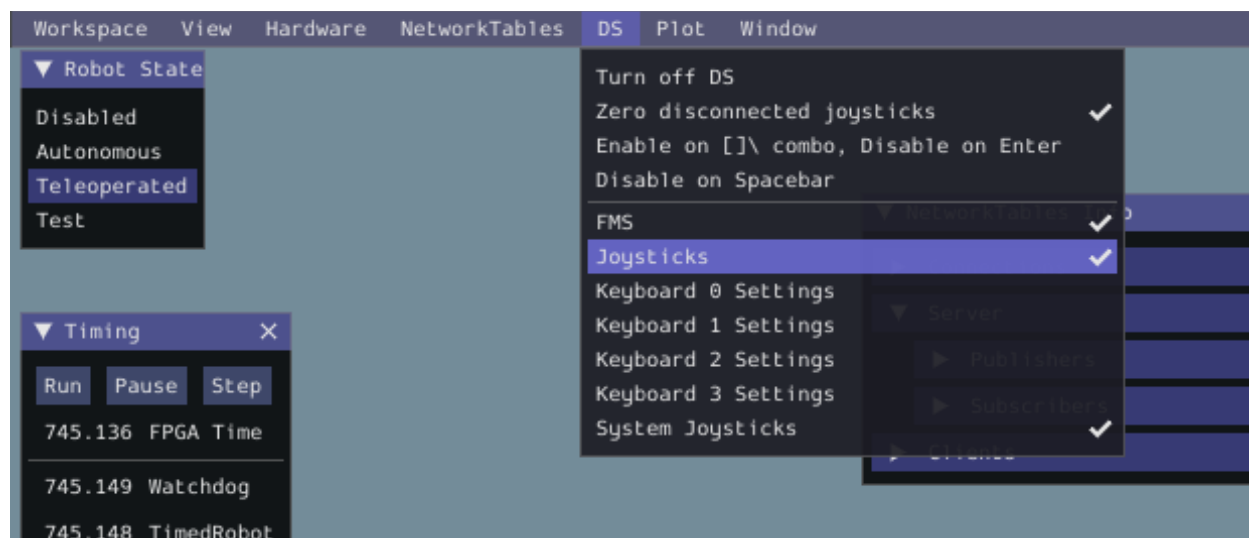


This will [compile your code](#) into a format that your computer can understand and then run it. Once it has finished compiling, you will need to make sure “Sim GUI” is ticked and “Use Real DriverStation” is un-ticked. Then click OK.

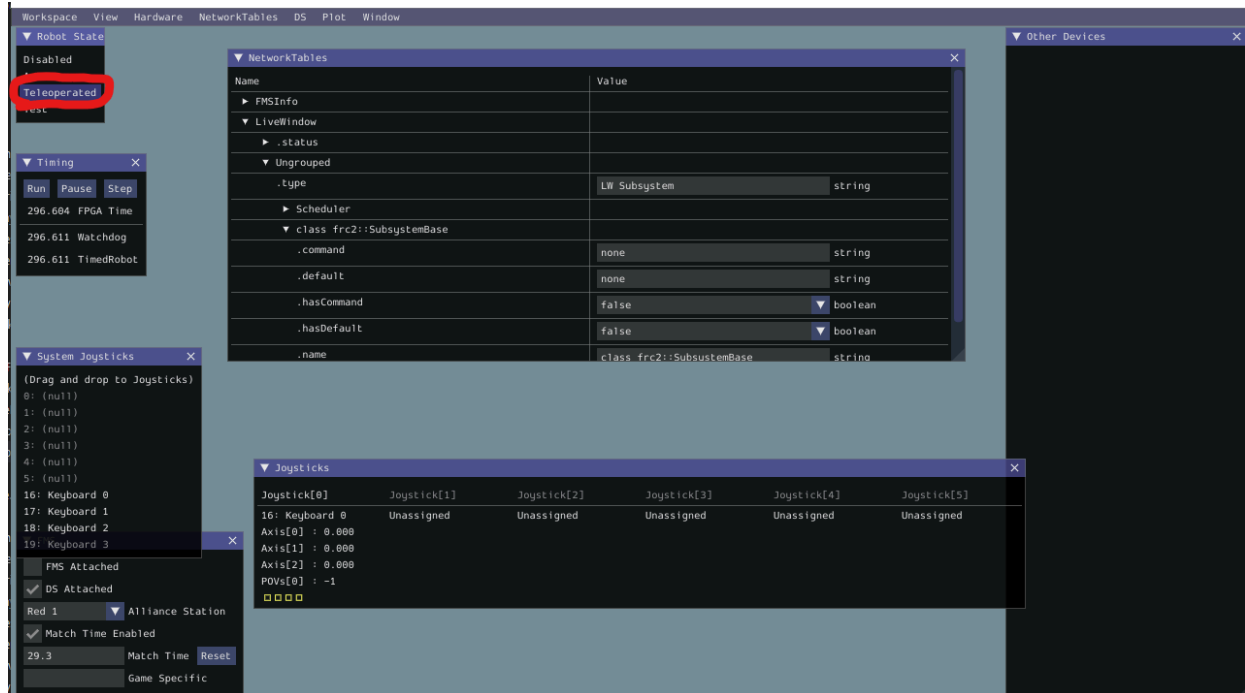


Check [here](#) for an overview of the simulator, you will need to follow the [Using keyboard as joystick](#) section of that tutorial if you don't have an xbox controller at home, or the [Adding a System Joystick to Joysticks](#) section if you do.

If you can't find a window or section mentioned in the instructions, you may need to add it from the title bar. For example, you will need to click DS > Joysticks to bring up the Joysticks window.



Once your joystick is added, click Teleoperated in the Robot State window at the top left.

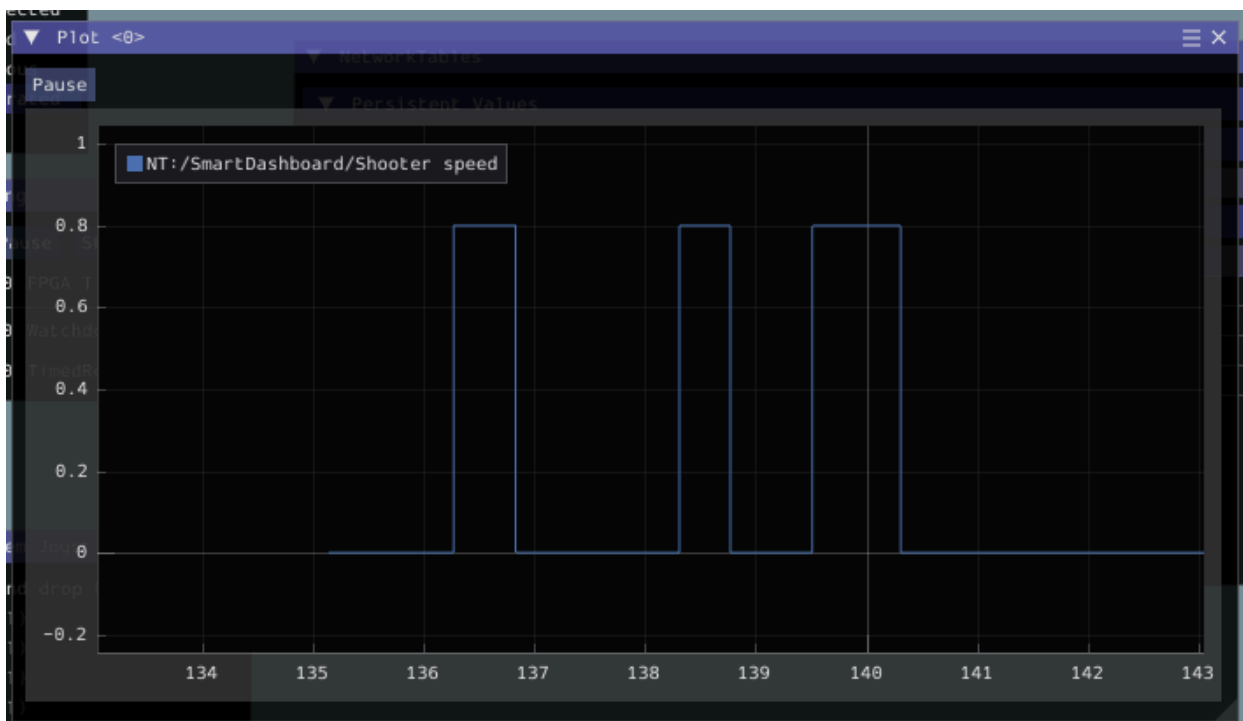


If you have a real Xbox controller plugged in, press A. If you are using a keyboard, click the button on your keyboard assigned to the A button on an XboxController, by default this is the Z key. Then scroll down in the NetworkTables view. Under SmartDashboard you will see your shooter power!

When the xbox controller's A button (probably the Z key on your keyboard) is pressed, the shooter power increases to 0.8 (80%) and when it is released, the power drops to 0.

NetworkTables	
▼ Persistent Values	
▼ Retained Values	
Name	Value
▼ Transitory Values	
Name	Value
▶ FMSInfo	
▶ LiveWindow	
▶ Shuffleboard	
▼ SmartDashboard	
Shooter speed	0.000000 double

Try following this [guide](#) to make a graph out of your shooter power:

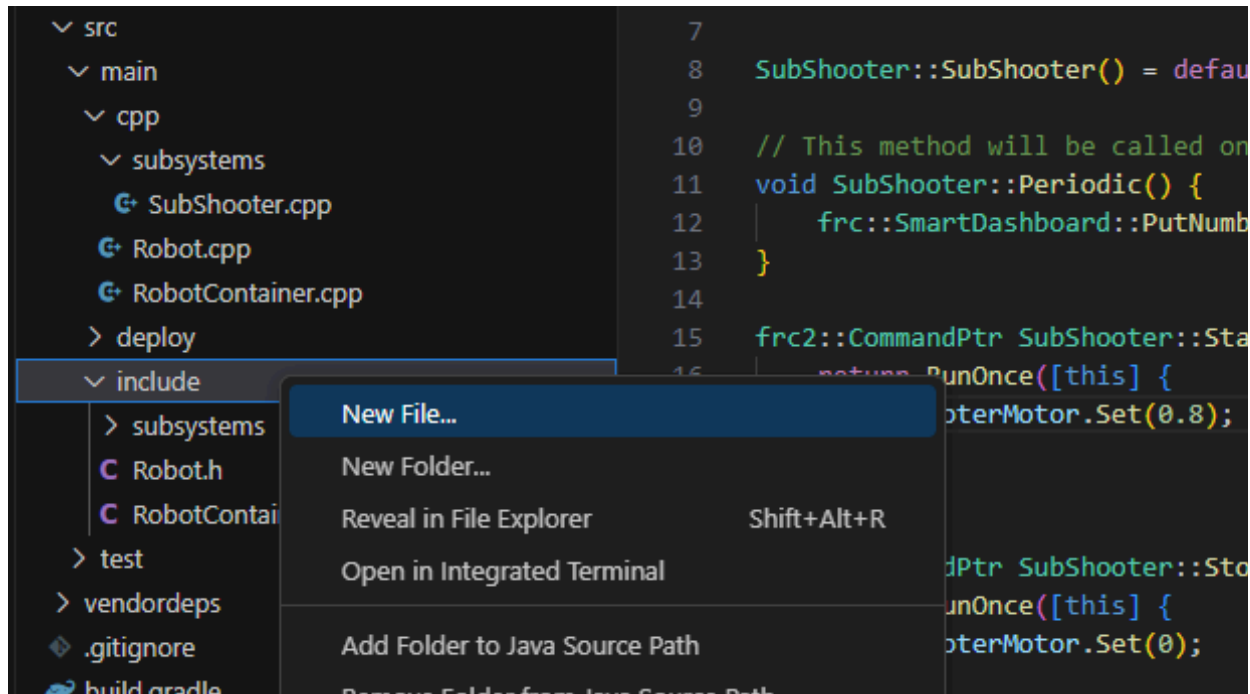


Moving IDs to constants.h

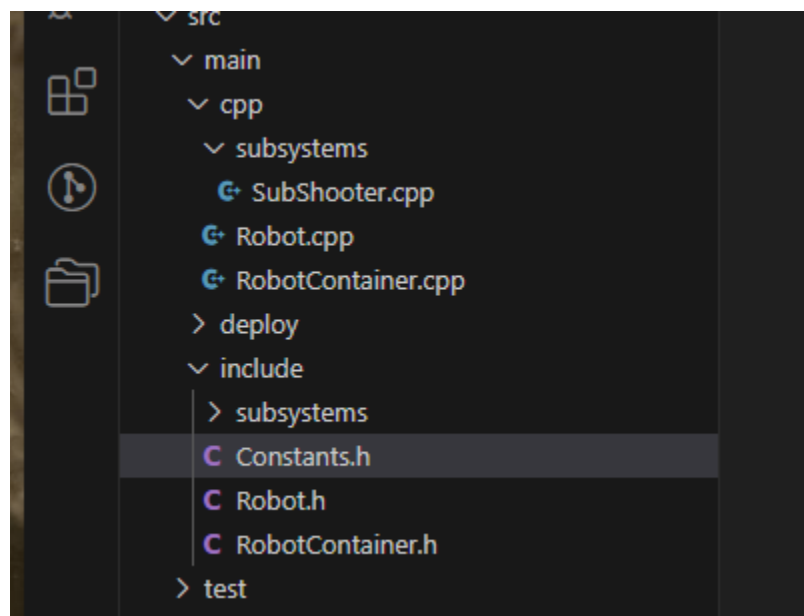
Remember when we created a Spark Flex with a CAN ID of 1? ([over here](#)). Writing the CAN ID inputs in-line as we create each electronic object gets very confusing when the program gets bigger, as it's easy to accidentally assign two things the same ID.

To solve this problem, we store all our electronic's IDs inside a single header file called Constants.h. This file doesn't have an associated body file since there is no logic or processes, just constant numbers.

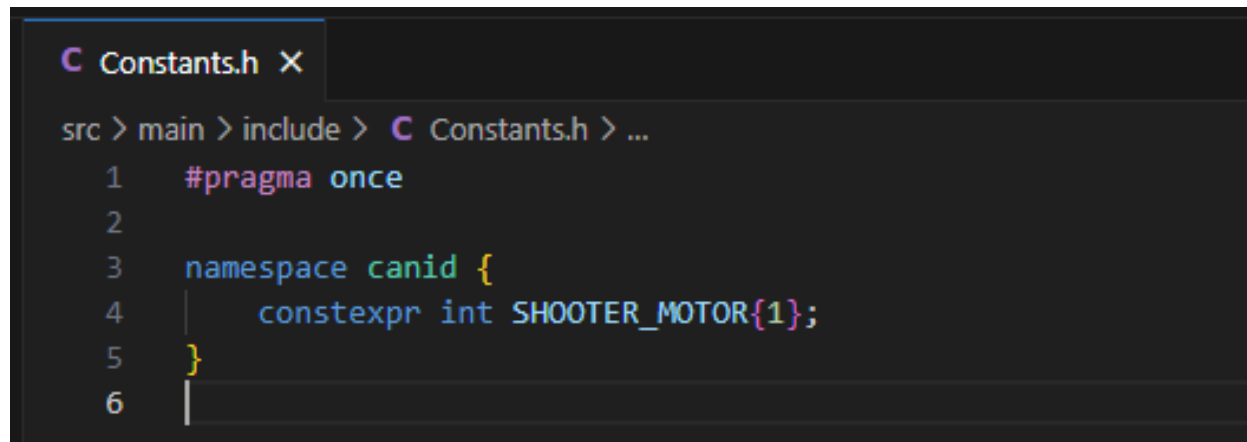
First, make Constants.h by right clicking on the src/main/include folder and selecting New File



Call the new file Constants.h.



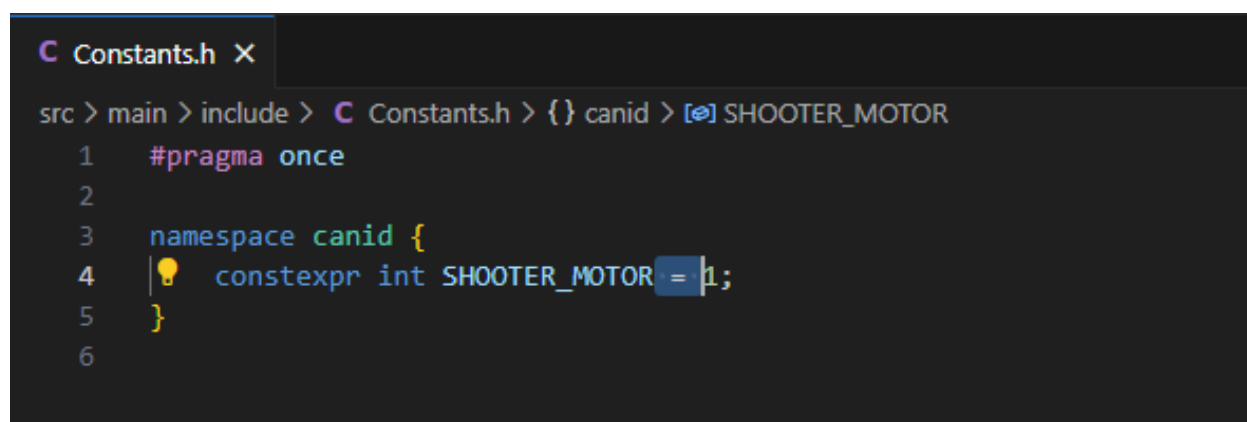
Let's move our Spark Max's CAN IDs to Constants.h to clean things up. We need to mark the file with `#pragma once` at the top so it can be included in multiple other places without breaking things, and then save the CAN ID in an object. This object will be of the integer (`int`) type, which is a whole number that does not have any decimal places. We also put the keyword `"constexpr"` in front to make it a constant, meaning no one will be allowed to change what value is stored in this object later on. As per our [Software Standards](#), let's name this constant with ALL_CAPS.



```
C Constants.h X
src > main > include > C Constants.h > ...
1  #pragma once
2
3  namespace canid {
4      constexpr int SHOOTER_MOTOR{1};
5  }
6
```

You'll often see simple types like integers that only store one value written with an `=` sign rather than `{ }` braces. Simple objects like this are more commonly known as variables or constants.

There is no functional difference between making variables with an `=` sign or `{ }` braces, but we'll switch to the more common `=` sign syntax for simple variables and constants from now on.



```
C Constants.h X
src > main > include > C Constants.h > {} canid > [e] SHOOTER_MOTOR
1  #pragma once
2
3  namespace canid {
4      constexpr int SHOOTER_MOTOR = 1;
5  }
6
```

We wrap this inside a namespace called `"canid"`, to make sure the name `"shooterMotor"` doesn't conflict with any other stuff that might have the

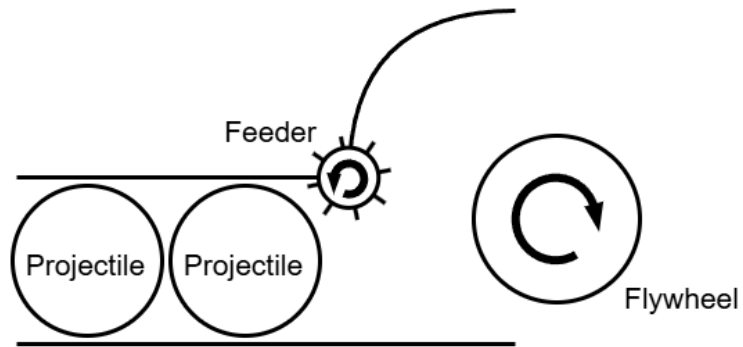
same name (like our SparkFlex object inside SubShooter!). We can now refer to this value anywhere in our program by #including Constants.h and writing "canid::SHOOTER_MOTOR". Let's use this in SubShooter:

```
C SubShooter.h X
src > main > include > subsystems > C SubShooter.h > SubShooter
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #pragma once
6
7 #include <frc2/command/SubsystemBase.h>
8 #include <rev/SparkFlex.h>
9 #include <frc2/command/Commands.h>
10 #include "Constants.h"
11
12 class SubShooter : public frc2::SubsystemBase {
13 public:
14     static SubShooter& GetInstance() {static SubShooter inst; return inst;};
15     SubShooter();
16
17     /**
18      * Will be called periodically whenever the CommandScheduler runs.
19      */
20     void Periodic() override;
21     frc2::CommandPtr SpinFlywheel();
22
23 private:
24     // Components (e.g. motor controllers and sensors) should generally be
25     // declared private and exposed only through public methods.
26     rev::spark::SparkFlex _shooterMotor{canid::SHOOTER_MOTOR, rev::spark::SparkLowLevel::MotorType::kBrushless};
27 };
28
```

After making any small changes like this, it's a good idea to test again. Run through [Testing in simulation](#) again to make sure everything still works.

Task: Adding a feeder

We now want to add a feeder subsystem to push balls into our shooter flywheel, so the balls are fed consistently and quickly. As the feeder rotates, it pushes a projectile ball into the flywheel.



Before looking at the solution below, try to implement a feeder subsystem in the same way we did for the shooter. It should have a Spark Flex motor controller, a command to run the feeder motor, and be bound to a button on the Xbox controller.

Test it in the simulator when ready.

Solution (Try yourself before looking!):

SubFeeder.h:	SubFeeder.cpp:
--------------	----------------

```

C SubFeeder.h X SubFeeder.cpp RobotContainer.h RobotContainer.cpp Constants.h
src > main > include > subsystems > C SubFeeder.h > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #pragma once
6
7 #include <frc2/command/SubsystemBase.h>
8 #include <frc2/command/Commands.h>
9 #include <rev/SparkFlex.h>
10 #include "Constants.h"
11
12 class SubFeeder : public frc2::SubsystemBase {
13 public:
14     static SubFeeder& GetInstance() {static SubFeeder inst; return inst;}
15     SubFeeder();
16
17     /**
18      * Will be called periodically whenever the CommandScheduler runs.
19      */
20     void Periodic() override;
21     frc2::CommandPtr Feed();
22
23 private:
24     // Components (e.g. motor controllers and sensors) should generally be
25     // declared private and exposed only through public methods.
26     rev::spark::SparkFlex _feederMotor(canid::FEEDER_MOTOR, rev::spark::SparkLowLevel::MotorType::kBrushless);
27 };
28

```

```

C SubFeeder.h SubFeeder.cpp X RobotContainer.h RobotContainer.cpp Const
src > main > cpp > subsystems > C SubFeeder.cpp > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "subsystems/SubFeeder.h"
6
7 SubFeeder::SubFeeder() = default;
8
9 // This method will be called once per scheduler run
10 void SubFeeder::Periodic() {}
11
12 frc2::CommandPtr SubFeeder::Feed() {
13     return StartEnd(
14         [this] {
15             _feederMotor.Set(1); // On Start
16         },
17         [this] {
18             _feederMotor.Set(0); // On End
19         }
20     );
21 }
22

```

Constants.h:

```

C SubFeeder.h SubFeeder.cpp RobotContainer.h RobotContainer.cpp
src > main > include > C Constants.h > ...
1 #pragma once
2
3 namespace canid {
4     constexpr int SHOOTER_MOTOR = 1;
5     constexpr int FEEDER_MOTOR = 2;
6 }
7

```

RobotContainer.cpp:

```

C SubFeeder.h SubFeeder.cpp RobotContainer.h RobotContainer.cpp X Constants.h
src > main > cpp > C RobotContainer.cpp > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "RobotContainer.h"
6
7 #include <frc2/command/Commands.h>
8 #include "subsystems/SubShooter.h"
9 #include "subsystems/SubFeeder.h"
10
11 RobotContainer::RobotContainer() {
12     ConfigureBindings();
13 }
14
15 void RobotContainer::ConfigureBindings() {
16     _controller.A().WhileTrue(SubShooter::GetInstance().SpinFlywheel());
17     _controller.B().WhileTrue(SubFeeder::GetInstance().Feed());
18 }
19
20 frc2::CommandPtr RobotContainer::GetAutonomousCommand() {
21     return frc2::cmd::Print("No autonomous command configured");
22 }
23

```

Command Groups

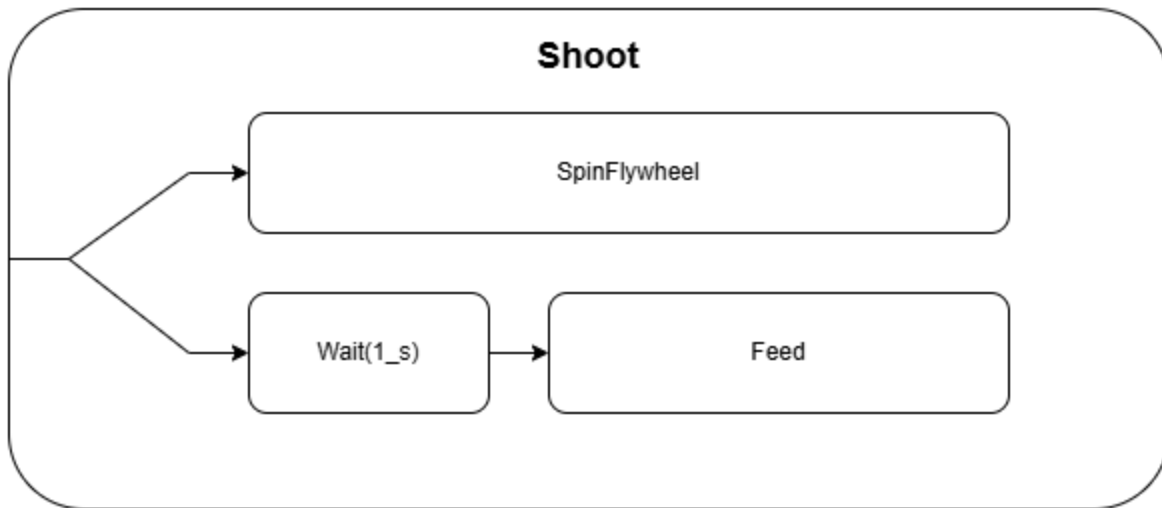
Making lots of small subsystems and commands is a good start, it's the kind of code we would write in week 1 of build season as we test different mechanisms. As the functionality of the robot becomes clearer, we normally want to start stringing commands together to automate functionality.

Commands are very powerful in how they can be composed together and altered to create new behaviors.

Let's compose some commands together in RobotContainer so that on a single button press, the shooter will start spinning, wait a moment for the wheel to get up to speed, then feed a ball in. Releasing the button should stop everything from moving.

```
SubFeeder.h SubFeeder.cpp RobotContainer.cpp X
src > main > cpp > RobotContainer.cpp > ConfigureBindings()
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #include "RobotContainer.h"
6
7 #include <frc2/command/Commands.h>
8
9 #include "subsystems/SubFeeder.h"
10 #include "subsystems/SubShooter.h"
11
12 RobotContainer::RobotContainer() { ConfigureBindings(); }
13
14 void RobotContainer::ConfigureBindings() {
15     ⚡
16     _controller.A().WhileTrue(
17         SubShooter::GetInstance().SpinFlywheel()
18         .AlongWith(
19             frc2::cmd::Wait(1_s)
20             .AndThen(SubFeeder::GetInstance().Feed())
21         )
22     );
23
24 }
25
26 frc2::CommandPtr RobotContainer::GetAutonomousCommand() {
27     return frc2::cmd::Print("No autonomous command configured");
28 }
29
```

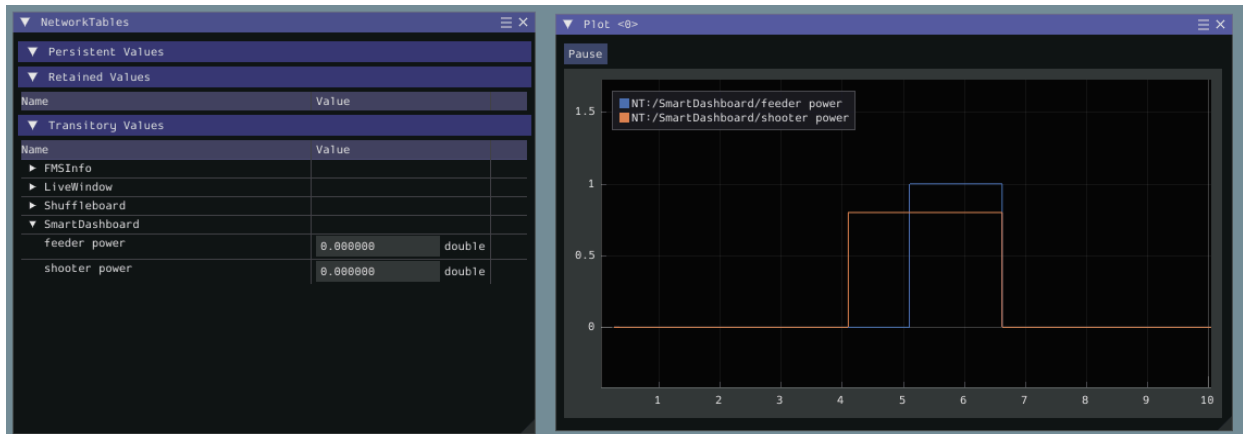
Notice the use of `frc2::cmd::Wait(1_s)`. This is a built-in command that can wait for a given amount of time. In this case we are requesting that it waits for 1 second. Here is a visual depiction of the command composition we have made:



A list of the built in command options [is available here](#).

Lets run the *WPILib: Simulate Robot Code* command again and see if it works.

Drag *feeder power* from the NetworkTables view into [our plot](#) and you should see that when you press the controller button, the new line jumps up in power 1 second after the shooter power. Make sure you hold the button for at least 1 second to see this happen.



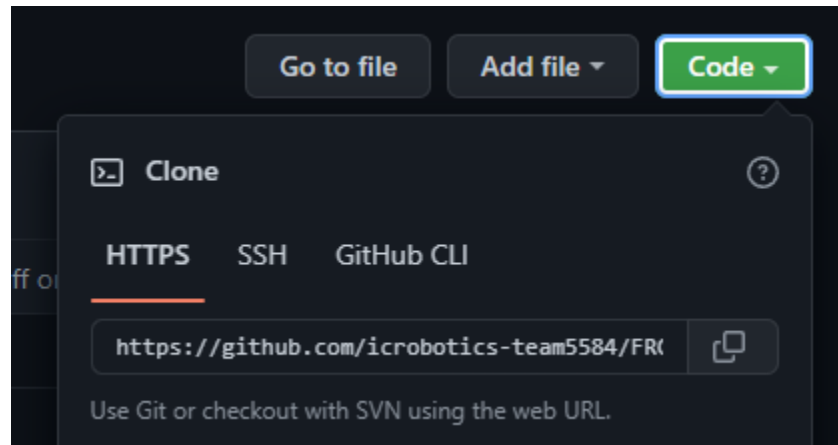
Getting ICRobotics Code

Install Git

Git is software that lets us save a history of versions of our code, share it within the team and work in parallel with each other. It's easy enough to install, grab it from <https://git-scm.com/downloads>. There are a lot of options on the installer, just leave them all as default.

Cloning a repository

1. Download ('clone') the latest IC software project from our GitHub page, as of March 2025 that would be [the Dwight project](#). If you don't have access, you'll need to make a GitHub account and ask the team to add you.
 - a. Click Code and copy the url in the text box:

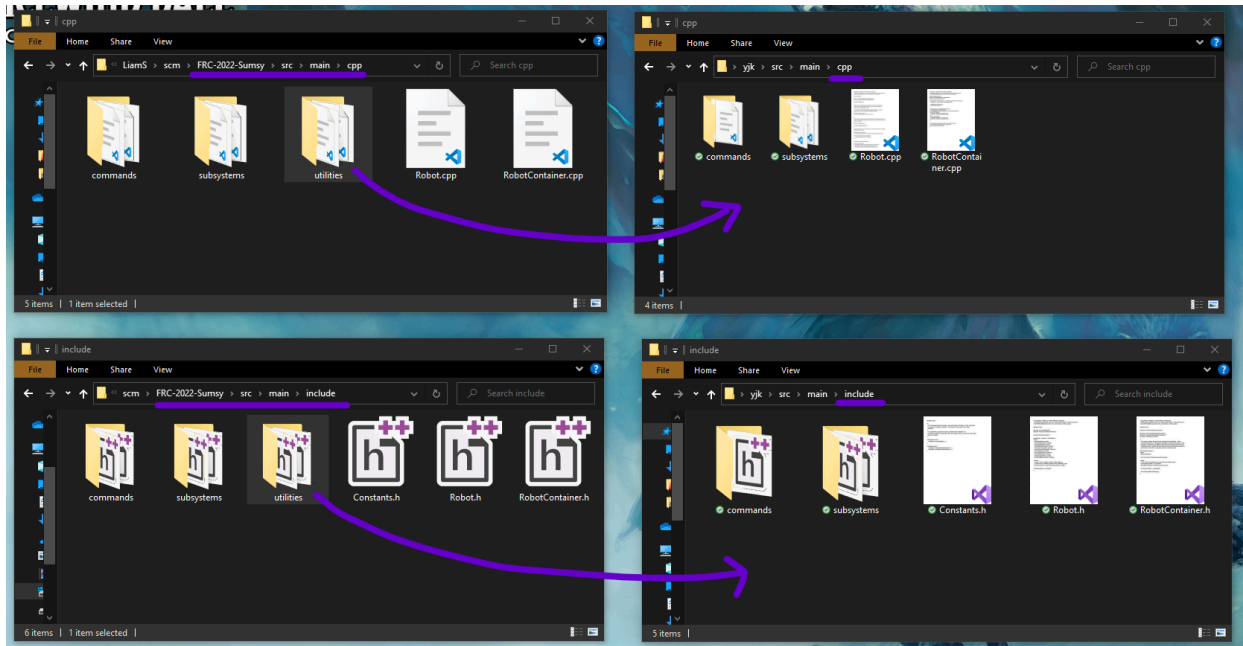


- b. Open WPILib VScode to a folder where you would like to store various IC software projects. We normally use C:\scm\icrobotics to store code projects, but it is up to you. (scm stands for source code management).
 - c. Open the command pallet by pressing Ctrl+Shift+P and run Git: Clone.
 - d. Enter the url copied from github and press enter

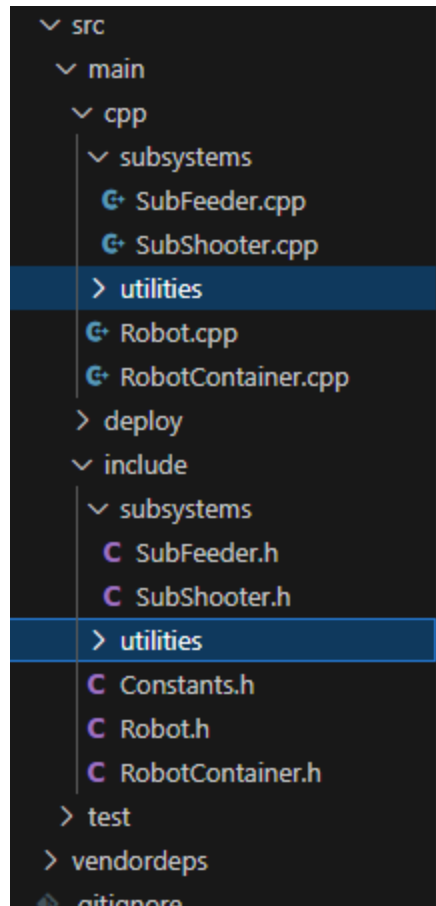
Cooler Code

Copying utilities from previous IC project

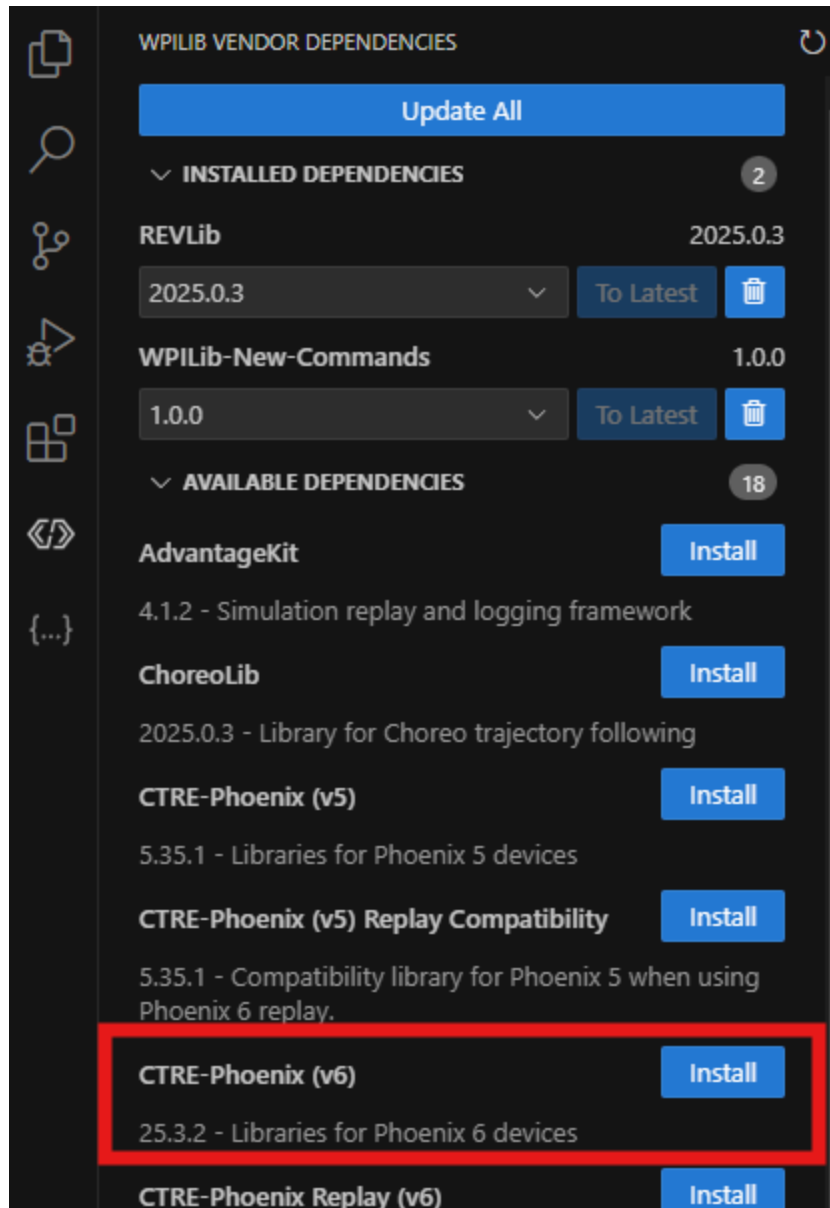
From the previous IC project you cloned in [this step](#), copy the utilities folders from /cpp and /include into the same places in your new project.



You should now have the utilities folders available in your project within VSCode:



Some of our utilities depend on another Vendor Library: CTRE Phoenix. This library is from Cross The Road Electronics and primarily lets us use their TalonFX Motor Controllers. Install CTRE-Phoenix (v6) from the WPILib Vendor Dependencies sidebar:



Control shooting speed with PIDF

In the Starter Code example, we set the shooter motor to spin at 80% power, but this does not guarantee that the shooter wheel will be spinning at the same speed all the time. When the battery gets low, or other systems use lots of power, the flywheel will slow down. This is a huge problem for shooters! Because a lower flywheel speed means the projectile won't shoot as far as it is supposed to and will most likely miss its target. We need

some smarter control systems that can keep the flywheel spinning at a constant velocity.

Introducing: Feedforward and Feedback control.

Read through [this explanation](#), skip over or google stuff you don't understand, the main thing is to have a go at the interactive examples. [This](#) is also a very good video on feedforward and feedback control.

As a summary, a Feed Forward (aka F or FF) controller uses our understanding of the physics of the system (friction, flywheel mass, etc) to calculate how much voltage to pump into a motor to make it spin at the speed we want. A Feedback controller (more specifically a PID feedback controller in most cases) reads from sensors and makes adjustments to compensate for any errors caused by disturbances that Feed Forward doesn't take into account (like a low battery, other subsystems using lots of voltage, or an opponent robot crashing into you at high speed). The best results are achieved when a combination of Feed Forward and Feedback is used.

Replace the SparkFlex in SubShooter with an ICSparkFlex, making sure to `#include "utilities/ICSparkFlex.h"` up the top. This will give us access to some new control functions.

```
SubFeeder.h SubFeeder.cpp RobotContainer.h RobotContainer.cpp SubShooter.h X
src > main > include > subsystems > C SubShooter.h > ...
1 // Copyright (c) FIRST and other WPILib contributors.
2 // Open Source Software; you can modify and/or share it under the terms of
3 // the WPILib BSD license file in the root directory of this project.
4
5 #pragma once
6
7 #include <frc2/command/SubsystemBase.h>
8 #include "utilities/ICSparkFlex.h"
9 #include <frc2/command/Commands.h>
10 #include "Constants.h"
11
12 class SubShooter : public frc2::SubsystemBase {
13 public:
14     static SubShooter& GetInstance() {static SubShooter inst; return inst;};
15     SubShooter();
16
17     /**
18      * Will be called periodically whenever the CommandScheduler runs.
19      */
20     void Periodic() override;
21     frc2::CommandPtr SpinFlywheel();
22
23 private:
24     // Components (e.g. motor controllers and sensors) should generally be
25     // declared private and exposed only through public methods.
26     ICSparkFlex _shooterMotor{canid::SHOOTER_MOTOR, 60_A};
27 };
28
```

Notice we now also give the shooter motor a different second input. It no longer takes a motor type (since the ICSparkFlex assumes a brushes motor), and instead takes a *current limit*. It is important to set current limits on motors for two main reasons:

- Drawing lots of current for a long time heats the motors up and cause damage
- Drawing lots of current from the battery means other subsystems on the robot may not have enough power to function.

60 Amps is a good starting spot for the REV motors we use.

Then change the SpinFlywheel() function to target a velocity of 1000 RPM (revolutions per minute).

```

14 }
15
16 frc2::CommandPtr SubShooter::SpinFlywheel() {
17     return StartEnd(
18         [this] {
19             _shooterMotor.SetVelocityTarget(1000_rpm); // On Start
20         },
21         [this] {
22             _shooterMotor.Set(0); // On End
23         });
24 }
25

```

The ICSparkFlex can put its own information onto the dashboard for us, so we can get rid of the SmartDashbaord line that sends the shooter speed, and instead send the whole object to the dashboard only once. We do that in a special function called the constructor. Every class (like our SubShooter class) has one function with the same name as itself, this is called the constructor and runs once when the class is created. Right now, the SubShooter's constructor is defaulted, so it does nothing:

```

C SubShooter.h  C Constants.h  SubShooter.cpp X  RobotContainer.cpp  W  ...
1  #include "subsystems/SubShooter.h"
2  #include <frc/smartdashboard/SmartDashboard.h>
3
4  SubShooter::SubShooter() = default;
5
6  // This method will be called once per scheduler run
7  void SubShooter::Periodic() {
8      frc::SmartDashboard::PutNumber("shooter speed", _shooterMotor.Get());
9  }

```

Let's change it to send our spark device. Note below that we use an & symbol in front of _shooterMotor. This is an operator in C++ that retrieves the memory address of an object. We use it because the SmartDashboard::PutData function doesn't want an object as input, it wants the memory address of an object (aka a [pointer or reference](#)).

```

6  #include <frc/smartdashboard/SmartDashboard.h>
7
8  SubShooter::SubShooter() {
9  |  frc::SmartDashboard::PutData("Shooter motor", &_shooterMotor);
10 | };
11
12 // This method will be called once per scheduler run
13 void SubShooter::Periodic() {
14 |
15 | }
16

```

Before we can simulate and see our flywheel spin up and reach a target speed, we need to define some physical characteristics of our shooter, since the way in which the motor spins depends on several physical factors like the mass of the shooter wheel (aka, flywheel).

To take care of our physics simulation, we will need to create a FlywheelSim object in SubShooter.h:

```

4
5  #pragma once
6
7  #include <frc2/command/SubsystemBase.h>
8  #include <frc2/command/Commands.h>
9  #include <frc/simulation/FlywheelSim.h>
10 #include <frc/system/plant/DCMotor.h>
11 #include <frc/system/plant/LinearSystemId.h>
12
13 #include "utilities/ICSparkFlex.h"
14 #include "Constants.h"
15
16 class SubShooter : public frc2::SubsystemBase {
17 private:
18 // Components (e.g. motor controllers and sensors) should generally be
19 // declared private and exposed only through public methods.
20 ICSparkFlex _shooterMotor{canid::SHOOTER_MOTOR, 60_A};
21
22 // Simulation
23 static constexpr double GEARING = 1.0;
24 static constexpr units::kilogram_square_meter_t MOI = 0.05_kg_sq_m;
25 static constexpr frc::DCMotor MOTOR_MODEL = frc::DCMotor::NeoVortex();
26 frc::LinearSystem<1,1> _flywheelSystem = frc::LinearSystemId::FlywheelSystem(MOTOR_MODEL, MOI, GEARING);
27 frc::sim::FlywheelSim _flywheelSim{ _flywheelSystem, MOTOR_MODEL};
28 };
29

```

Above, we are dealing with a few new object types:

- GEARING is stored as a double, which is a number with decimal places,
- FLYWHEEL_MOI is a kilogram_square_meter_t, which is the first number we have seen with its units specified. It's good to use real world units whenever possible to reduce the risk of conversion mistakes or accidentally using the wrong units. In this case, kg_sq_m measures an object's [*Moment of Inertia*](#) (*MOI*), or how difficult it is to spin. Objects with higher MOI require more energy to spin.
- MOTOR_MODEL is a frc::DCMotor, which is a mathematical model of how a motor performs. It can calculate how much torque (aka rotational force) would be generated given some input voltage and current. We set it equal to a model of a NEO Vortex motor, which is the type of motor most commonly attached to Spark Flex motor controllers.
- _flywheelSystem is a frc::LinearSystem. This is a type that ties together our motor model, MOI and gearing. It can calculate the speed and acceleration of the shooter wheel given some input voltage and current. You might notice the <1,1,1>, this denotes that the system has one state that we are trying to control (speed), one input (voltage) and one output (speed, again).
- _flywheelSim is an object of the type frc::sim::FlywheelSim. This represents a simulated flywheel, it uses the flywheelSystem to help run its physics simulation of the mechanism.

We need somewhere to run our physics simulation, so let's make a special function that will run periodically during simulation (not when the program is running on a real robot). This function is aptly named SimulationPeriodic(). It already exists in the base subsystem (a place that defines all the things that every Subsystem can do). To use SimulationPeriodic(), we need to override it and give it a definition of our own, then the internal scheduler will run it every 20 milliseconds during simulation.

```

14 class SubShooter : public frc2::SubsystemBase {
15     public:
16         SubShooter();
17         static SubShooter& GetInstance() {
18             static SubShooter inst;
19             return inst;
20         }
21
22         void Periodic() override; // called every 20ms
23         void SimulationPeriodic() override; // called every 20ms in simulation only
24
25         frc2::CommandPtr SpinFlywheel();
26
27     private:

```

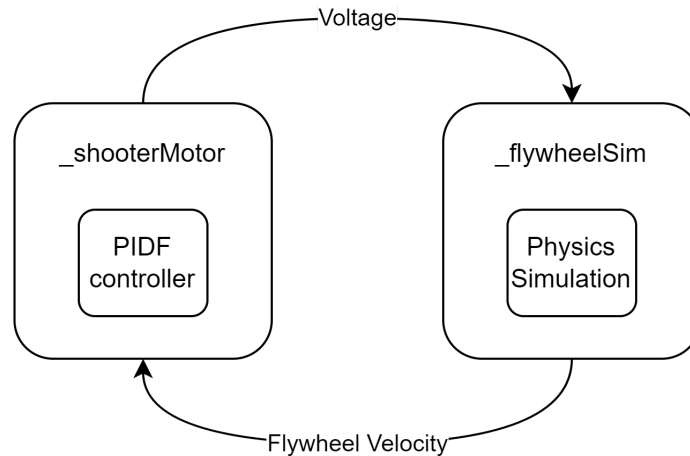
Jump to the SubShooter body file (ending in .cpp) and give the SimulationPeriodic() function a definition.

```

16 void SubShooter::SimulationPeriodic() {
17     // Calculate how much voltage the Spark Motor Controller is applying to the motor and store
18     // this result in an object of type units::volt_t called volts.
19     units::volt_t volts = _shooterMotor.CalcSimVoltage();
20
21     // Simulate putting that much voltage into the flywheel system for
22     // 20 milliseconds (the duration of one loop of our whole program)
23     _flywheelSim.SetInputVoltage(volts);
24     _flywheelSim.Update(20_ms);
25
26     // Calculate and save the angular velocity (aka speed) that the flywheel is moving. Since the
27     // type name of this object would be quite long (units::radians_per_second_t), we can use the
28     // auto keyword to let C++ figure out the type itself.
29     auto velocity = _flywheelSim.GetAngularVelocity();
30
31     // Send the angular velocity back to the Spark Max.
32     _shooterMotor.IterateSim(velocity);
33 }
34
35

```

The comment lines above (starting with //) attempt to explain what's going on. In summary, we ask the Spark how much voltage it is putting into the motor, send that into the physics simulation, then ask the physics simulation how fast the wheel is spinning and put that back into the Spark so it can do its feedback control (PID).



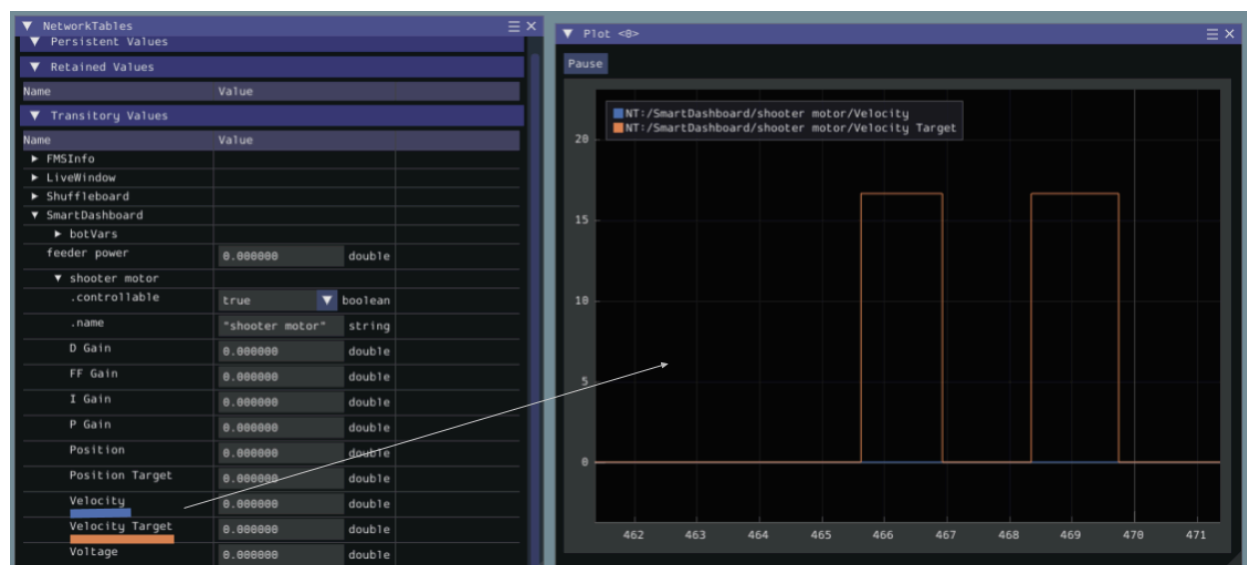
The code above looks like a lot, but once you get some practice with saving objects, passing inputs to functions and converting between different units, the code can be simplified to something like this:

```

16
17 void SubShooter::SimulationPeriodic() {
18     // Simulate the flywheel mechanism
19     _flywheelSim.SetInputVoltage(_shooterMotor.CalcSimVoltage());
20     _flywheelSim.Update(20_ms);
21     _shooterMotor.IterateSim(_flywheelSim.GetAngularVelocity());
22 }
23

```

Running the code in sim and graphing the target velocity and actual velocity:



We can see that when we hold the shoot button, the target velocity jumps up to ~17 turns per second (~= the 1000 rpm that we asked for). But the actual velocity does not change at all.

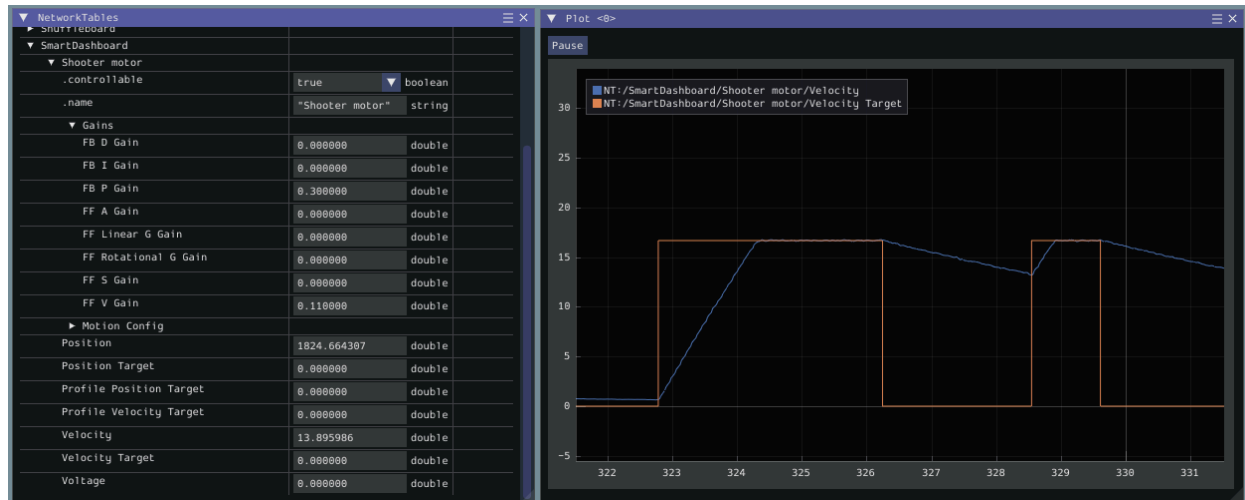
This is because we have not set out PIDF gains, as described [here](#).

Try playing with the PIDF gains shown in the NetworkTables to get the velocity to closely match the target.

A good method is to set everything to zero, then incrementally double Feedforward V until you reach the speed you want, then increase P until you get there quickly without overshooting or oscillating. I and D can often stay at zero for a well made flywheel.

▼ NetworkTables		
▶ Shutterboard		
▼ SmartDashboard		
▼ Shooter motor		
.controllable	true ▼	boolean
.name	"Shooter motor"	string
▼ Gains		
FB D Gain	0.000000	double
FB I Gain	0.000000	double
FB P Gain	0.000000	double
FF A Gain	0.000000	double
FF Linear G Gain	0.000000	double
FF Rotational G Gain	0.000000	double
FF S Gain	0.000000	double
FF V Gain	0.000000	double
▶ Motion Config		
Position	0.000000	double
Position Target	0.000000	double
Profile Position Target	0.000000	double
Profile Velocity Target	0.000000	double
Velocity	0.000000	double
Velocity Target	0.000000	double
Voltage	0.000000	double

With some tuning, we can get the actual velocity to match the target.



We then need to save those PID and FeedForward gains in the code. We do that in the constructor as part of the subsystem's setup. PID gains go into a config that is sent to the Spark to run onboard that motor controller (which reduces sensing delays). Feedforward parameters are set separately with direct function calls.

```

7
8 SubShooter::SubShooter() {
9     frc::SmartDashboard::PutData("Shooter motor", &_shooterMotor);
10
11     rev::spark::SparkBaseConfig config;
12     config.closedLoop.P(0.3);
13     _shooterMotor.AdjustConfig(config);
14     _shooterMotor.SetFeedforwardVelocity(0.11_v / 1_tps); // Requires 0.11 volts to drive at 1 turn per second
15 };
16
17 // This method will be called once per scheduler run
18 void SubShooter::Periodic() {

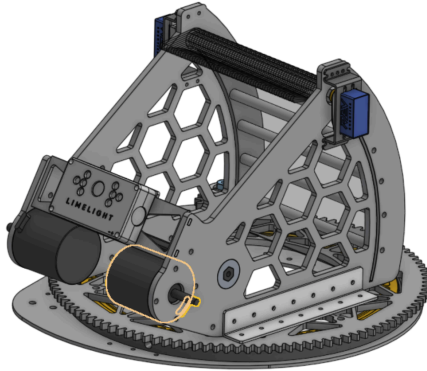
```

So above we are setting $P = 0.3$ and $V = 0.11$. All other gains (feedback gains of I and D, feedforward gains of S, G, A) remain at the default of zero.

Simulate again and you should see the gains populated correctly.

Adding a turret with position control

Shooters with velocity control are good, but they're even better when we can aim them independently of the drivebase. Let's add a turret to aim the shooter! See how this turret has a big toothed gear on the bottom?



That's because another motor attaches to it to make the whole thing rotatable. There is another interactive example for a turret [here](#). Take a look through that and try to tune the example turret.

Make a new Subsystem for the turret. In its header (.h) file, make an ICSparkFlex to control the rotation.

```
src > main > include > subsystems > C SubTurret.h > SubTurret
1  #pragma once
2
3  #include <frc2/command/SubsystemBase.h>
4  #include "utilities/ICSparkFlex.h"
5  #include "Constants.h"
6
7  class SubTurret : public frc2::SubsystemBase {
8  public:
9      SubTurret();
10
11     void Periodic() override;
12
13     private:
14     ICSparkFlex _turretMotor{canid::TURRET_MOTOR, 60_A};
15 };
16
```

And add the GetInstance() function:

```

13
14 class SubTurret : public frc2::SubsystemBase {
15     public:
16         SubTurret();
17         static SubTurret& GetInstance() {
18             static SubTurret inst;
19             return inst;
20         }
21

```

Lets also declare a function to create a command that rotates the turret to a given angle. This command will be different to the ones we've made before because it takes an input: the target angle. Until now, all our commands do the same thing every time they are run. When you add inputs (aka parameters) to functions, you can use that information to change how the function operates. Give our function inputs by writing out the type of the input and its name inside the () parentheses. We will take in an input of the type `units::degree_t` to represent an angle and call it "angle".

```

3 #include <frc2/Command/SubsystemBase.h>
4 #include <frc2/command/Commands.h>
5 #include "utilities/ICSparkFlex.h"
6 #include "Constants.h"
7
8 class SubTurret : public frc2::SubsystemBase {
9     public:
10         SubTurret();
11         static SubTurret& GetInstance() {static SubTurret inst; return inst;}
12
13         void Periodic() override;
14
15         // Commands
16         frc2::CommandPtr TurnTo(units::degree_t angle); // Rotate the turret to angle. 0_deg = forward
17
18     private:

```

Since our turret motor will likely go through a [gearing reduction](#) before moving the turret (to increase its strength and decrease its speed), we also want to note down the gear ratio so we can command the turret itself to particular angles rather than the motor shaft. We'll use a gear ratio of 20:1.

```

16   frc2::CommandPtr TurnTo(units::degree_t angle); // Rotate the turret to
17
18   private:
19       // Electronics
20       ICSparkFlex _turretMotor{canid::TURRET_MOTOR, 60_A};
21
22       // Constants
23       static constexpr double GEARING = 20;
24   };
25

```

Now let's jump to the body file and give the TurnTo() command a definition. We need to convert the target angle given as input from **degrees of our turret** into **revolutions of our motor**. That can be done in two ways. Option 1 is every time you set a target, multiply it by the gear ratio, which would look like:

```

28   frc2::CommandPtr SubTurret::TurnTo(units::degree_t angle) {
29       return Run([this, angle] {
30           // Option 1: Multiply target angle by gear ratio
31           _turretMotor.SetPositionTarget(angle * GEARING);
32       });
33   }
34

```

Or alternatively, you can tell the Spark to use a conversion factor once, and it will do the multiplying for you behind the scenes whenever you use it. Let's undo the multiplication (option 1) and stick with option 2.

```

7
8   SubTurret::SubTurret() {
9       // Option 2: Set a conversion factor in the spark's config
10      rev::spark::SparkBaseConfig config;
11      config.encoder.PositionConversionFactor(1.0 / GEARING);
12      config.encoder.VelocityConversionFactor(1.0 / GEARING / 60.0); // divide by 60 to get rotations per second not RPM
13      _turretMotor.AdjustConfig(config);
14   };
15
16   // This method will be called once per schedule run
17
24
25   frc2::CommandPtr SubTurret::TurnTo(units::degree_t angle) {
26       return Run([this, angle] {
27           // Option 1: Multiply target angle by gear ratio
28           _turretMotor.SetPositionTarget(angle);
29       });
30   }
31

```

Notice we set the conversion factor to 1.0/GEARING, not just to GEARING. The Spark Max expects its conversion factor to be input gear teeth / output gear teeth, whereas the WPILib simulation objects expect output teeth / input teeth so we'll have to invert one or the other.

Just like we did for the flywheel, we need to set our PIDF gains in the subsystem constructor.

```
7 SubTurret::SubTurret() {
8     // Option 2: Set a conversion factor in the spark's config
9     rev::spark::SparkBaseConfig config;
10    config.encoder.PositionConversionFactor(1.0 / GEARING);
11    config.encoder.VelocityConversionFactor(1.0 / GEARING / 60.0); // divide by
12    config.closedLoop.P(0.1);
13    config.closedLoop.D(0.1);
14    _turretMotor.AdjustConfig(config);
15 };
```

This sets $P=0.1$ and $D=0$. Since we have set no feedforward gains, this is effectively a feedback only controller.

Then, we can make the physics simulation object like we did for the flywheel except this time we use a `DCMotorSim` instead of a `FlywheelSim`, because a `DCMotorSim` also simulates position whereas the `FlywheelSim` only simulates velocity.

```
2
3 #include <frc2/command/SubsystemBase.h>
4 #include <frc2/command/Commands.h>
5 #include <frc/simulation/DCMotorSim.h>
6 #include <frc/system/plant/DCMotor.h>
7 #include <frc/system/plant/LinearSystemId.h>
8 #include "utilities/ICSparkFlex.h"
9 #include "Constants.h"
10
24
25 // Constants
26 static constexpr double GEARING = 20;
27 static constexpr units::kilogram_square_meter_t MOI = 0.05_kg_sq_m;
28
29 // Simulation
30 static constexpr frc::DCMotor MOTOR_MODEL = frc::DCMotor::NeoVortex();
31 frc::LinearSystem<2,1,2> _turretSystem = frc::LinearSystemId::DCMotorSystem(MOTOR_MODEL, MOI, GEARING);
32 frc::sim::DCMotorSim _turretSim(_turretSystem, MOTOR_MODEL);
33
34 };
35
```

To use the `DCMotorSim` physics simulation object, we'll make a `SimulaitonPeriodic()` function.

Header file:

```

15 public:
16     SubTurret();
17
18     void Periodic() override; // called every 20ms
19     void SimulationPeriodic() override; // called every 20ms in sim only
20
21     // Commands
22     frc2::CommandPtr TurnTo(units::degree_t angle); // Rotate turret to angle. 0_deg = forward.
23
24
25 private:

```

Body file:

```

18
19 void SubTurret::SimulationPeriodic() {
20     _turretSim.SetInputVoltage(_turretMotor.CalcSimVoltage());
21     _turretSim.Update(20_ms);
22     _turretMotor.IterateSim(_turretSim.GetAngularVelocity());
23 }

```

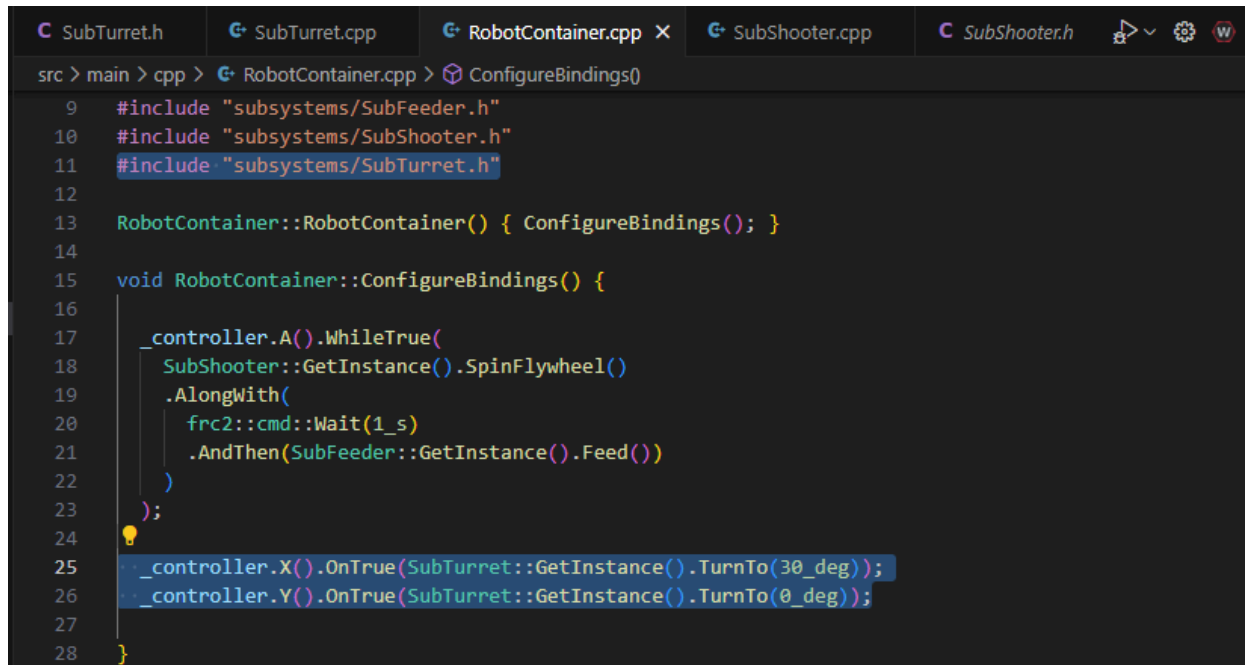
Next, send the turret motor to the dashboard.

```

4
5 #include "subsystems/SubTurret.h"
6 #include <frc/smartdashboard/SmartDashboard.h>
7
8 SubTurret::SubTurret() {
9     rev::spark::SparkBaseConfig config;
10     config.encoder.PositionConversionFactor(1.0 / GEARING);
11     config.encoder.VelocityConversionFactor(1.0 / GEARING / 60.0); // divide
12     config.closedLoop.P(0.1);
13     config.closedLoop.D(0.1);
14     _turretMotor.AdjustConfig(config);
15
16     frc::SmartDashboard::PutData("Turret motor", &_turretMotor);
17 };

```

Finally, add a couple of buttons in RobotContainer.cpp to drive the turret to different angles. Because we set the command up with inputs (aka parameters), we can give our TurnTo command different values inside the () parentheses when we call it.



```
9  #include "subsystems/SubFeeder.h"
10 #include "subsystems/SubShooter.h"
11 #include "subsystems/SubTurret.h"
12
13 RobotContainer::RobotContainer() { ConfigureBindings(); }
14
15 void RobotContainer::ConfigureBindings() {
16
17     _controller.A().WhileTrue(
18         SubShooter::GetInstance().SpinFlywheel()
19         .AlongWith(
20             frc2::cmd::Wait(1_s)
21             .AndThen(SubFeeder::GetInstance().Feed())
22         )
23     );
24
25     _controller.X().OnTrue(SubTurret::GetInstance().TurnTo(30_deg));
26     _controller.Y().OnTrue(SubTurret::GetInstance().TurnTo(0_deg));
27
28 }
```

Test in simulation again and see if you can tune the turret's PID gains to make the turret reach its setpoint (aka target) faster.

Checking when you're on target

Soon we'll get into making a command that further automates the robot's shooting actions. Before we can get to that though, we will need to be able to query our Subsystems to check whether they are on target. This will allow us to wait for the turret to be at its target angle and the shooter to be at its target velocity before feeding a ball in.

These functions will be a little different to the ones we've made before since they will need to give us back a result (aka "return" a result).

Specifically, we need an `AtTarget()` function that returns **true** when its subsystem is on target and **false** when it is not. True and false values can be stored in a simple type called a boolean (aka "bool"). To make a function that returns an object of type bool, write *bool* where you would normally write *frc2::CommandPtr* (or *void* in some cases) in the function declaration and definition.

Let's make our `AtTarget()` functions, starting with SubShooter:

SubShooter Header:

```
C SubTurret.h  C SubShooter.h x  SubTurret.cpp  RobotContainer.cpp  SubShooter.cpp  Cor w [
src > main > include > subsystems > C SubShooter.h > SubShooter > AtTarget()
15 public:
16 SubShooter();
17 static SubShooter& GetInstance() {
18     static SubShooter inst;
19     return inst;
20 }
21
22 void Periodic() override; // called every 20ms
23 void SimulationPeriodic() override; // called every 20ms in simulation only
24 bool AtTarget(); // Check if flywheel has reached target velocity
25
26 frc2::CommandPtr SpinFlywheel();
27
28 private;
```

SubShooter Body:

```
28
29 bool SubShooter::AtTarget() {
30     if (_shooterMotor.OnVelTarget(20_rpm)) { // 20 RPM tolerance
31         return true;
32     } else {
33         return false;
34     }
35 }
36
```

Notice how we are returning true if we are within 20 RPM of the target rather than only returning true when we are exactly on the target. This is because it will be almost impossible for our system to reach the target exactly, it might overshoot by as little as 0.001 RPM, so we want to build in some tolerance. A tolerance of 20 RPM is pretty good for an FRC shooter and should keep our projectiles firing reasonably accurately.

While we're here, we can also simplify this code a little:

Since `motor.OnVelTarget()` will effectively become true or false when the program is running, we can replace our if else statement with this:

```
28
29 bool SubShooter::AtTarget() {
30     return _shooterMotor.OnVelTarget(20_rpm);
31 }
32
```

Now let's make an AtTarget() function for the Turret, using a tolerance of 1 degree.

SubTurret Header:

```
13
14 class SubTurret : public frc2::SubsystemBase {
15     public:
16         SubTurret();
17     static SubTurret& GetInstance() {
18         static SubTurret inst;
19         return inst;
20     }
21
22     void Periodic() override; // called every 20ms
23     void SimulationPeriodic() override; // called every 20ms in sim only
24     bool AtTarget(); // check if turret is at target angle
25
26     // Commands
```

SubTurret Body:

```
27
28     bool SubTurret::AtTarget() {
29         return _turretMotor.OnPosTarget(1_deg);
30     }
31
```

Great, we have our AtTarget() functions that allow us to ask the subsystems whether they are on target or not. Lets send the results of those functions to the dashboard periodically to test that they work.

SubTurret body:

```
14 // This method will be called once per scheduler run
15 void SubTurret::Periodic() {
16     frc::SmartDashboard::PutBoolean("Turret on target", AtTarget());
17 }
```

SubShooter body:

```
14 // This method will be called once per scheduler run
15 void SubShooter::Periodic() {
16     frc::SmartDashboard::PutBoolean("Shooter on target", AtTarget());
17 }
```

Test these out in sim.

Then, let's move on to improving the automation of our commands using the new boolean functions.

Command Automation

The drivers have decided that it would be nice if the robot could turn its turret to the target angle, speed up the shooter wheel to the right velocity and then fire a projectile as soon as the systems are ready all on one button press.

We've only talked about commands enough to get us going so far, let's dive a bit deeper. Commands are robot actions that run over time. There are three stages to each command, a start, middle and end (more formally known as Initialize, Execute and End).

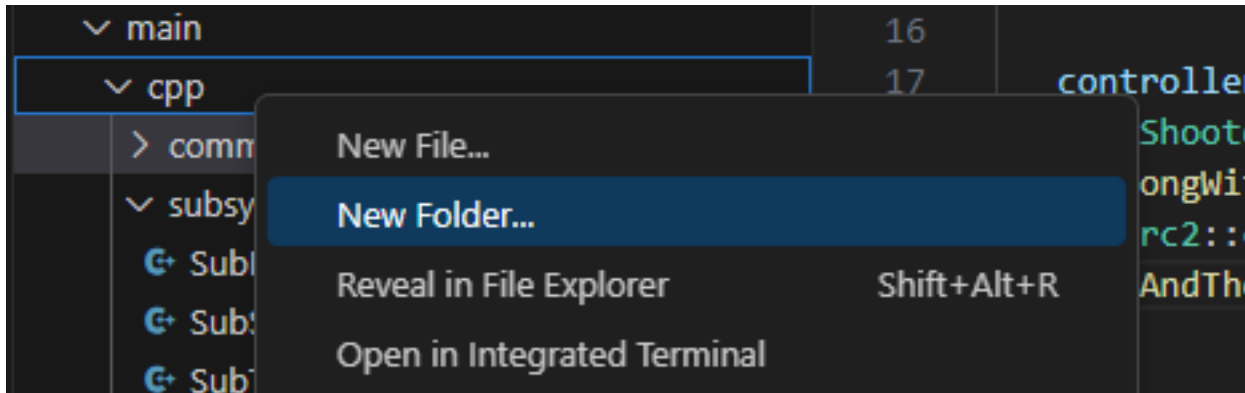
When a command runs,

1. The Initialize stage runs once,
2. Then the Execute stage runs in a loop over and over again for as long as the command is active,
3. Then the End stage runs a single time when the command is stopped.

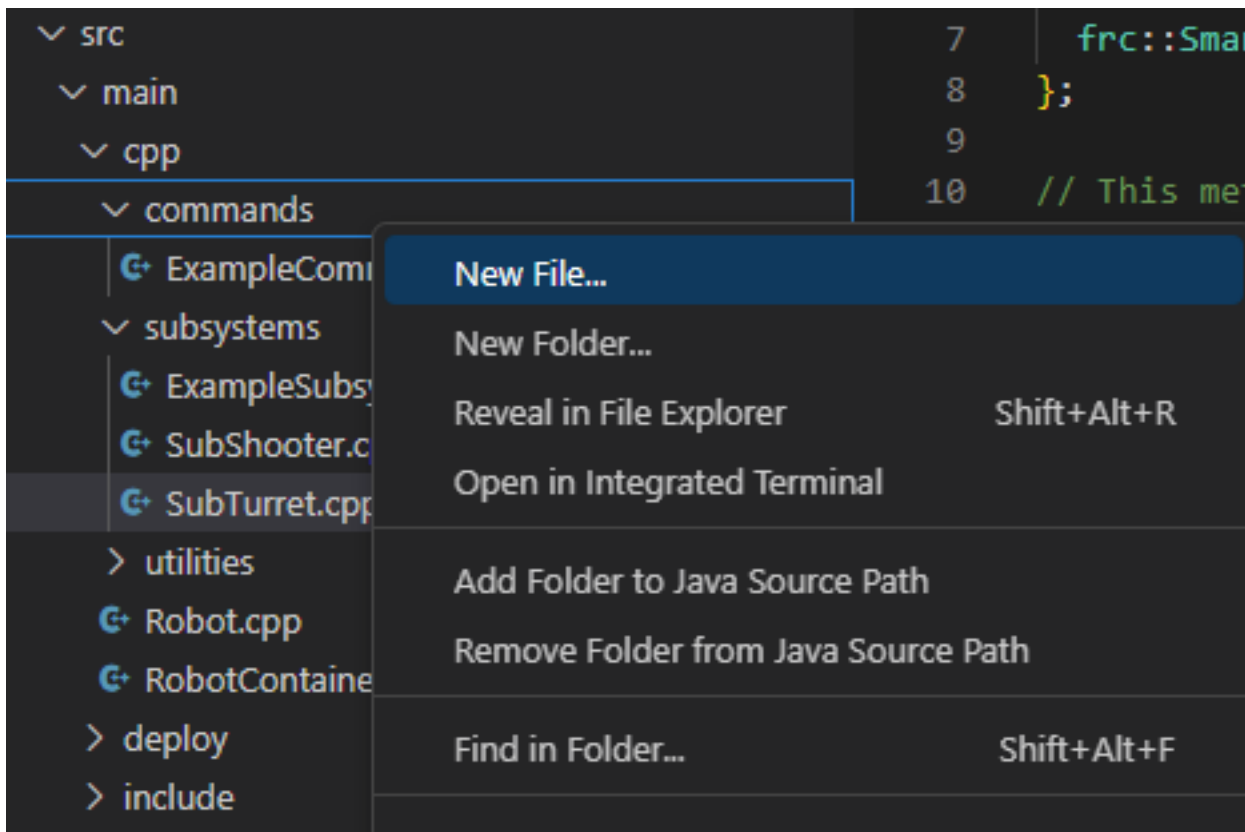
A more detailed description of commands and their stages is available [here](#).

There are many tools for making common types of commands that utilize the various stages, a full list is available [here](#). It includes “**factories**” that help make commands, and “**decorators**” that alter how a command behaves. Having a good understanding of this list and what command-making-tools are available is very helpful for making complex actions in a simple way. In the following example, we will use the Parallel, Sequence and WaitUntil command factories.

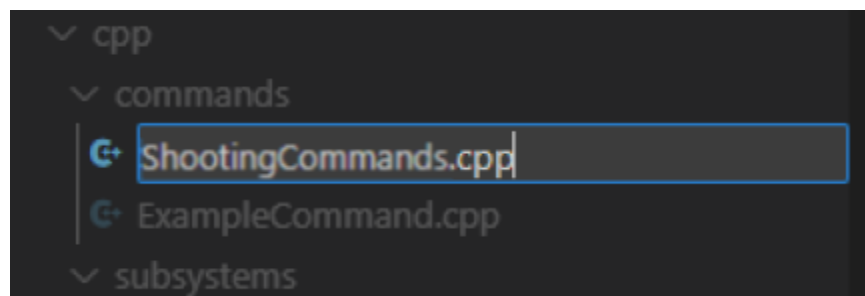
Let's make a new file to hold our commands now that things are getting more complicated. Right click the src/main/cpp/ folder and make a new commands sub folder.



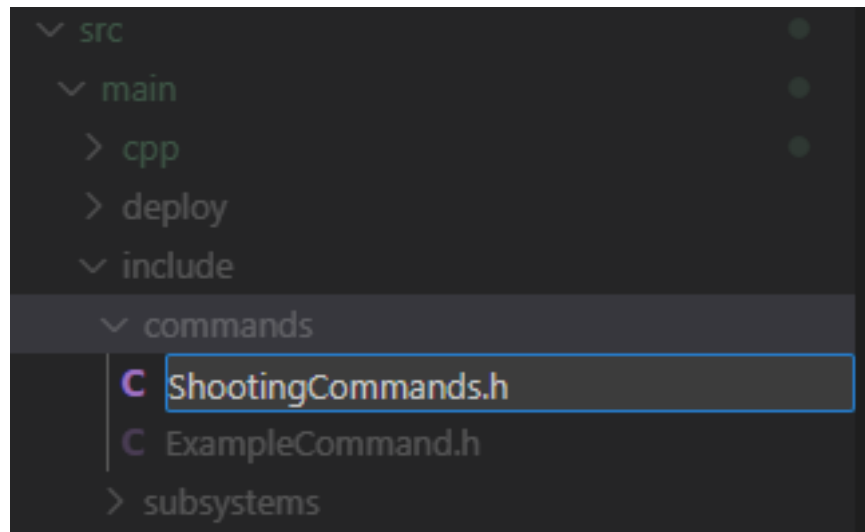
Inside that, make a new file.



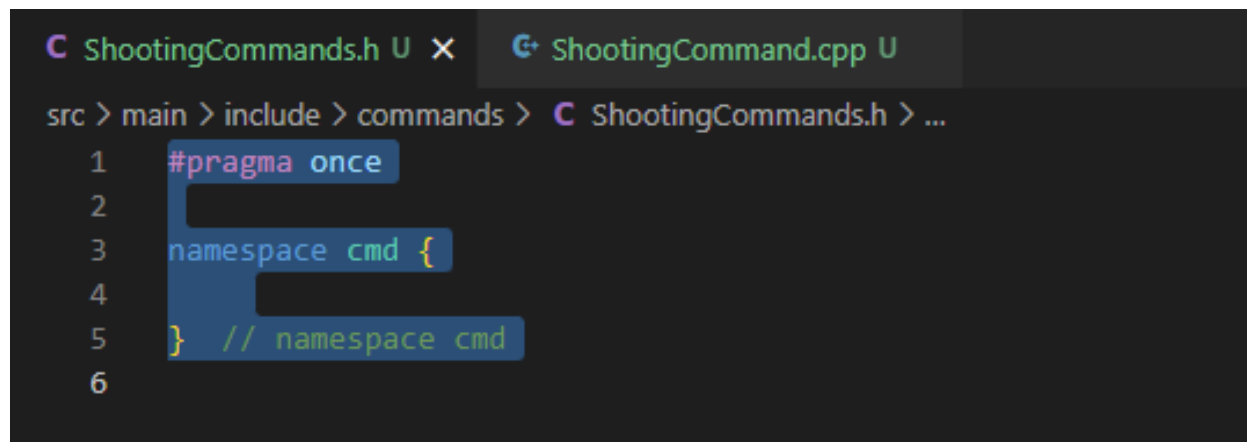
We're thinking of storing all our commands related to shooting in this file, so let's name it ShootingCommands.cpp



Do the same thing in src/main/include to create the ShootingCommands.h header file.



We store our commands in namespaces to make sure their names don't clash with any other names in our project. Make a cmd namespace in ShootingCommand.h like so:



We also want to #include <frc2/command/commands.h> like we did in RobotContainer.

```
C ShootingCommands.h U X  ShootingCommand.cpp U
src > main > include > commands > C ShootingCommands.h > ...
1  #pragma once
2
3  #include <frc2/command/commands.h>
4
5  namespace cmd {
6
7  } // namespace cmd
8
```

Lets add the namespace to the body too, don't forget to include its header.

```
C ShootingCommands.h U  ShootingCommand.cpp U X
src > main > cpp > commands > ShootingCommand.cpp > ...
1  #include "commands/ShootingCommands.h"
2
3  namespace cmd {
4
5  }
6
7
```

Now in the header, we'll declare a new shoot sequence command:

```
C ShootingCommands.h X  ShootingCommands.cpp
src > main > include > commands > C ShootingCommands.h > {} cmd > ShootSequence()
1  #pragma once
2
3  #include <frc2/command/commands.h>
4
5  namespace cmd {
6  frc2::CommandPtr ShootSequence();
7  }
```

And in the body file, define it like so. We will also need to #include the subsystems we want to use.

```
C ShootingCommands.h  ShootingCommands.cpp X  SubTurret.cpp  SubFeeder.cpp  St > v ⚙ W
src > main > cpp > commands > ShootingCommands.cpp > ...
1  #include "commands/ShootingCommands.h"
2
3  #include "subsystems/SubFeeder.h"
4  #include "subsystems/SubShooter.h"
5  #include "subsystems/SubTurret.h"
6
7  namespace cmd {
8  frc2::CommandPtr ShootSequence() {
9      return frc2::cmd::Parallel(
10         SubShooter::GetInstance().SpinFlywheel(),
11         SubTurret::GetInstance().TurnTo(30_deg),
12         frc2::cmd::Sequence(
13             frc2::cmd::WaitUntil([] {return SubShooter::GetInstance().AtTarget();}),
14             frc2::cmd::WaitUntil([] {return SubTurret::GetInstance().AtTarget();}),
15             SubFeeder::GetInstance().Feed()
16         )
17     );
18 }
19 } // namespace cmd
20
21
```

Since we use a lot of command factories inside the `frc2::cmd` namespace in command files like our `ShootingCommands.cpp` file, we can add a `using` line to simplify things a bit and avoid needing to type `frc2::cmd` so many times.

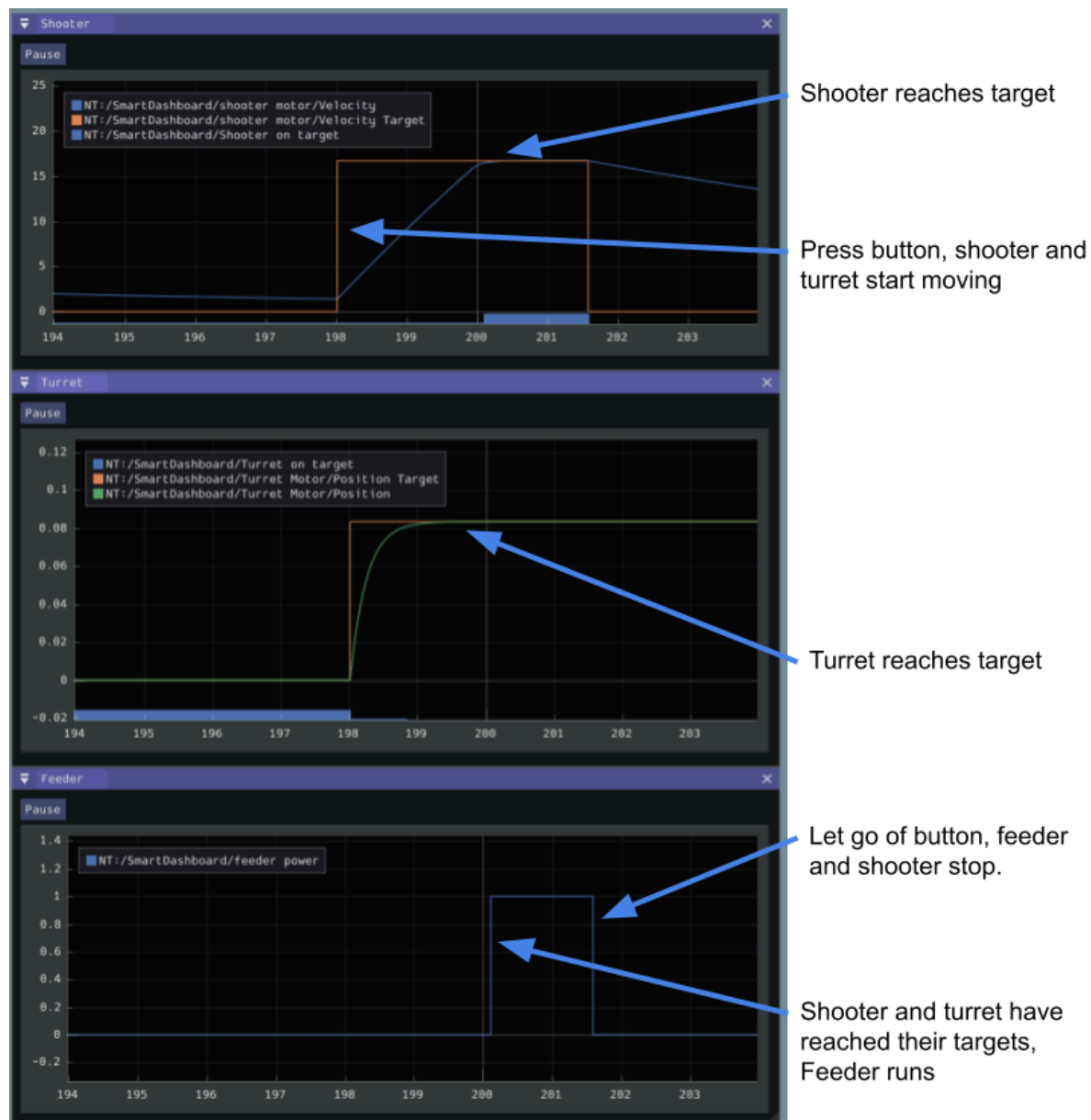
Note that [using namespace can be dangerous](#), don't do it in any header files and keep it to commands files only.

```
C ShootingCommands.h  ShootingCommands.cpp  SubTurret.cpp  SubFeeder.cpp  St
src > main > cpp > commands > ShootingCommands.cpp > {} cmd
1  #include "commands/ShootingCommands.h"
2
3  #include "subsystems/SubFeeder.h"
4  #include "subsystems/SubShooter.h"
5  #include "subsystems/SubTurret.h"
6
7  namespace cmd {
8
9  using namespace frc2::cmd;
10
11  frc2::CommandPtr ShootSequence() {
12      return Parallel(
13          SubShooter::GetInstance().SpinFlywheel(),
14          SubTurret::GetInstance().TurnTo(30_deg),
15          Sequence(
16              WaitUntil([] {return SubShooter::GetInstance().AtTarget();}),
17              WaitUntil([] {return SubTurret::GetInstance().AtTarget();}),
18              SubFeeder::GetInstance().Feed()
19          )
20      );
21  }
22  } // namespace cmd
23
```

In RobotContainer.cpp, let's include our new commands file and add ShootSequence to a button.

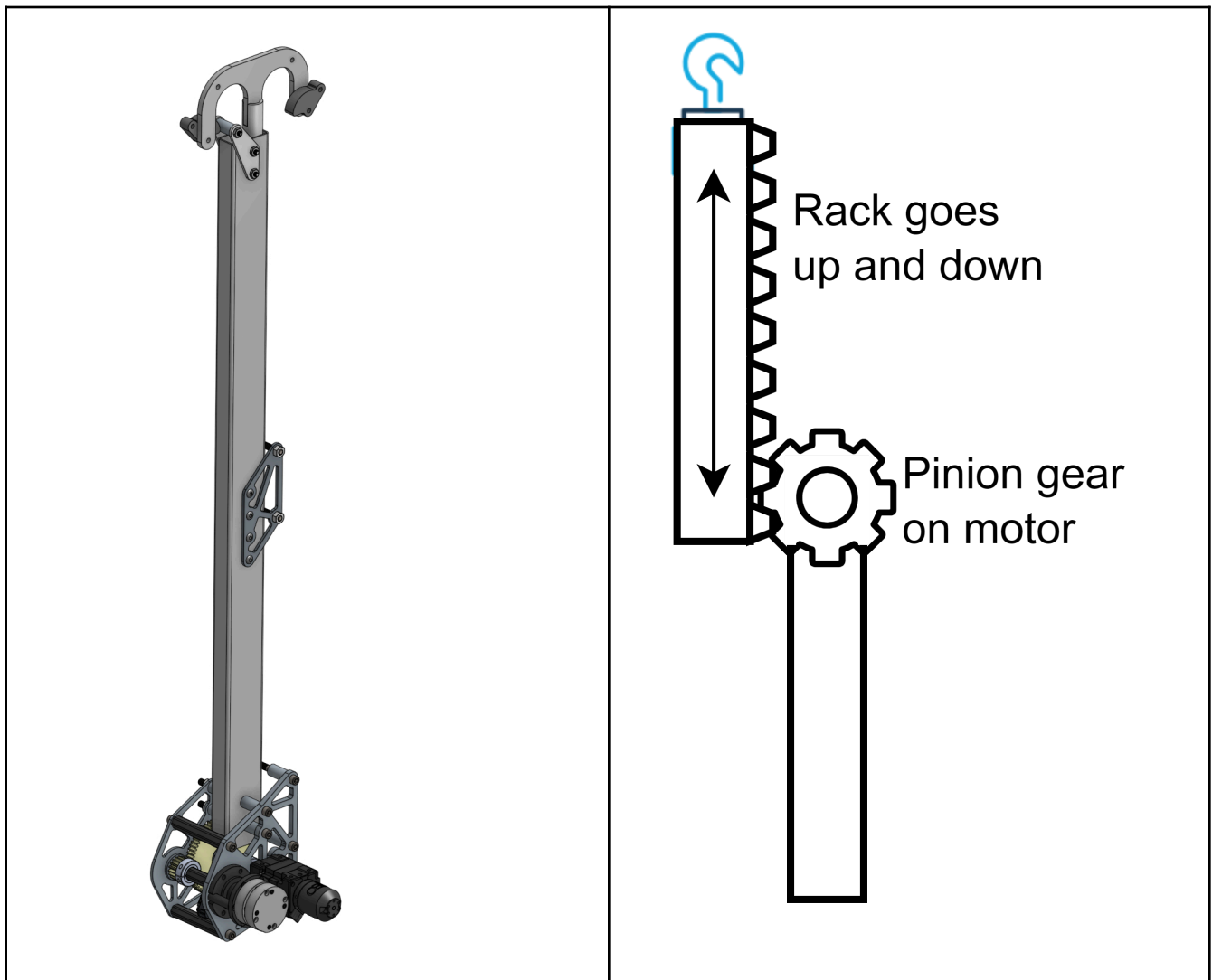
```
C ShootingCommands.h  ShootingCommands.cpp  RobotContainer.cpp  X
src > main > cpp > RobotContainer.cpp > ConfigureBindings()
1  // Copyright (c) 2017 FRC 6328, All Rights Reserved.
2  // Open Source Software; you can modify and/or share it under the terms of
3  // the WPILib BSD license file in the root directory of this project.
4
5  #include "RobotContainer.h"
6
7  #include <frc2/command/Commands.h>
8
9  #include "subsystems/SubFeeder.h"
10 #include "subsystems/SubShooter.h"
11 #include "subsystems/SubTurret.h"
12
13 #include "commands/ShootingCommands.h"
14
15 RobotContainer::RobotContainer() { ConfigureBindings(); }
16
17 void RobotContainer::ConfigureBindings() {
18     _controller.A().WhileTrue(cmd::ShootSequence());
19
20     _controller.X().OnTrue(SubTurret::GetInstance().TurnTo(30_deg));
21     _controller.Y().OnTrue(SubTurret::GetInstance().TurnTo(0_deg));
22
23 }
24
```

Testing our new ShootSequence command in the simulator and making some more plots for all the subsystems, we can see the sequence play out:



Task: Create a climber subsystem

Almost every FRC game has some form of climbing challenge. The robots must lift themselves up onto a bar, rope, giant coat hanger, or whatever else the Game Design Committee comes up with. One of the most common ways to climb onto a bar is by elevating a hook that latches onto the bar and then retracting the hook thereby pulling the robot off the ground.



Imagine rotating the pinion gear in the diagram above. This will push the rack and its hook up or down.

An elevator like this can be controlled with just one motor on the pinion gear. Try making a subsystem and commands to control a climbing mechanism like the one above.

Think about what electronic components the subsystem has, then what functions it can perform.

This is an excellent chance to practice using PID position control like we did for the turret.

An elevator can be simulated in the same way we did the shooter and turret, but using an [ElevatorSim](#) instead of a [FlywheelSim](#) like we did for the flywheel or a [DCMotorSim](#) like we did for the turret.

Ninja Code

The following topics are very common, but out of the scope of this guide. Links are provided in each topic to dive a little further, but self research will be required to figure them out. Importantly, many members of the IC Software team are across these topics and can teach you one on one.

Making a swerve drive base

- WPILib example code:
<https://github.com/wpilibsuite/allwpilib/tree/main/wpilibcExamples/src/main/cpp/examples/SwerveDrivePoseEstimator>
- Previous IC swerve drive base subsystem header:
[FRC-2024-GAUSS/src/main/include/subsystems/SubDrivebase.h at main · icrobotics-team5584/FRC-2024-GAUSS \(github.com\)](#)
- Previous IC swerve drive base subsystem body:
[FRC-2024-GAUSS/src/main/cpp/subsystems/SubDrivebase.cpp at main · icrobotics-team5584/FRC-2024-GAUSS \(github.com\)](#)
- Previous IC swerve module header:
[FRC-2024-GAUSS/src/main/include/utilities/SwerveModule.h at main · icrobotics-team5584/FRC-2024-GAUSS \(github.com\)](#)
- Previous IC swerve module body:
[FRC-2024-GAUSS/src/main/cpp/utilities/SwerveModule.cpp at main · icrobotics-team5584/FRC-2024-GAUSS \(github.com\)](#)

Knowing where you are on the field

This is super useful for both the autonomous and teleoperated periods, it makes it possible to aim at and drive to arbitrary points on the field.

- WPILib example code:
<https://github.com/wpilibsuite/allwpilib/tree/main/wpilibcExamples/src/main/cpp/examples/SwerveDrivePoseEstimator>

- Photonvision guide on adding vision measurements:
<https://docs.photonvision.org/en/latest/docs/examples/poseest.html>
!

Autonomous

- Zero to autonomous youtube series (written in Java, but theories still apply to C++): <https://youtu.be/0Xi9yb1IMyA>
- Pathplanner (The tool we use to generate autonomous paths, which replaces WPILib trajectories): <https://pathplanner.dev/home.html>

WPILib Advanced Controls

- [Advanced Controls – FIRST Robotics Competition documentation \(wpilib.org\)](https://wpilib.org/advanced-controls)